# BÁO CÁO DỰ ÁN CHUYÊN NGÀNH 1 (PBL6)

## ĐỀ TÀI: HỆ THỐNG TRUY VẤN BÀI HÁT QUA TIẾNG NGÂN NGA (QUERY-BY-HUMMING) DỰA TRÊN CONFORMER-BYTEHUM V6.2

## TÓM TẮT DỰ ÁN (ABSTRACT)

Báo cáo trình bày toàn diện quá trình nghiên cứu, thiết kế kiến trúc học sâu, tối ưu hóa xử lý tín hiệu và triển khai thực tế của Hệ thống Truy vấn Bài hát qua Tiếng ngân nga (Query-by-Humming - QBH) phiên bản Conformer-ByteHum V6.2.

Khác biệt với các hệ thống truyền thống chỉ dựa trên mạng CNN nông và phổ Chroma CQT 12 bins gập quãng tám (như khảo sát trong QBH.pdf), dự án đề xuất giải pháp đột phá kết hợp 4 trụ cột công nghệ:

1. **Xử lý tín hiệu cao cấp**: Sử dụng Meta HTDemucs v4 bóc tách 100% Vocal Stems sạch từ bản Master Studio, kết hợp phép biến đổi Full CQT 96 bins (8 quãng tám) và chuỗi Data Augmentation đặc thù (Pitch Shift ± 12 bán âm, Reverb, Harmonic Noise).
2. **Kiến trúc mô hình Conformer-ByteHum 9.91M tham số**: Kết hợp khả năng nắm bắt nhịp điệu cục bộ của Depthwise Convolution và ngữ cảnh giai điệu toàn cục của Multi-Head Self-Attention (MHSA). Tích hợp cơ chế Attentive Statistics Pooling (ASP) cùng bộ ba mô đun tinh hoa của ByteHum (ICASSP 2024): LAT (Learned Alignment Transformer) với MaxMean Pooling, LAC (Learned Alignment Classifier) với L2 Prototypes, và GRL (Gradient Reversal Layer) huấn luyện đối kháng miền.
3. **Thuật toán tìm kiếm hai chặng**: Top-K Quantile Pooling 85% Cosine Similarity lọc nhanh trên bộ nhớ đệm In-RAM Feature Cache, kết hợp Fused Distance (75% Metric Score + 25% LAT Score).
4. **Hạ tầng phân phối đám mây Cloudinary CDN**: Lưu trữ và cung cấp luồng phát trực tiếp (Direct Streaming URL) cho 300 bài hát với độ trễ tìm kiếm toàn trình chỉ 315 ms trên GPU.

Kết quả thực nghiệm trên tập kiểm thử độc lập khẳng định sự vượt trội toàn diện: chỉ số **MRR đạt 0.5086** (tăng +35.0%), **Recall@1 đạt 42.23%** (tăng gấp 2.53 lần), **Recall@5 đạt 60.50%** và **Recall@10 đạt 77.10%**.

## DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

| Từ viết tắt | Thuật ngữ tiếng Anh đầy đủ | Diễn giải ý nghĩa |
|---|---|---|
| **QBH** | Query-by-Humming | Hệ thống truy vấn bài hát qua tiếng ngân nga, hát chay hoặc huýt sáo |

| Từ viết tắt | Thuật ngữ tiếng Anh đầy đủ | Diễn giải ý nghĩa |
|---|---|---|
| **MIR** | Music Information Retrieval | Lĩnh vực khoa học công nghệ Truy xuất Thông tin Âm nhạc |
| **CQT** | Constant-Q Transform | Biến đổi Q hằng số với thang tần số logarit |
| **STFT** | Short-Time Fourier Transform | Biến đổi Fourier thời gian ngắn với dải tần số tuyến tính |
| **MHSA** | Multi-Head Self-Attention | Cơ chế Tự chú ý đa đầu trong kiến trúc Transformer |
| **ASP** | Attentive Statistics Pooling | Khối gộp thống kê kỳ vọng và độ lệch chuẩn có trọng số tự chú ý |
| **LAT** | Learned Alignment Transformer | Bộ căn chỉnh giai điệu cục bộ thông qua Cross-Attention MaxMean |
| **LAC** | Learned Alignment Classifier | Bộ phân loại căn chỉnh dựa trên vector tâm cụm nguyên mẫu L2 |
| **GRL** | Gradient Reversal Layer | Lớp đảo ngược dấu gradient phục vụ huấn luyện đối kháng miền |
| **MRR** | Mean Reciprocal Rank | Xếp hạng nghịch đảo trung bình của bài hát đúng |
| **CDN** | Content Delivery Network | Mạng lưới phân phối nội dung đa phương tiện toàn cầu (Cloudinary) |
| **RMS** | Root Mean Square | Giá trị căn bậc hai trung bình bình phương (đo năng lượng âm thanh) |
| **BPM** | Beats Per Minute | Số phách nhịp trong mỗi phút (tốc độ bài hát) |

# CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG TRUY VẤN BÀI HÁT QUA TIẾNG NGÂN NGA (QUERY-BY-HUMMING)

## 1.1. Đặt vấn đề và Tính cấp thiết của đề tài

Trong thời đại số hóa và bùng nổ dữ liệu truyền thông đa phương tiện, các nền tảng phát nhạc trực tuyến như Spotify, Apple Music, YouTube Music đang lưu trữ hàng trăm triệu bản nhạc với thể loại, ngôn ngữ và phong cách biểu diễn vô cùng phong phú. Việc hỗ trợ người dùng tìm kiếm bài hát một cách trực quan, tức thời trở thành bài toán cốt lõi của ngành công nghệ Truy xuất Thông tin Âm nhạc (Music Information Retrieval - MIR).

Thông thường, người dùng tìm kiếm bài hát qua siêu dữ liệu văn bản (tên bài hát, tên nhạc sĩ, ca sĩ thể hiện) hoặc lời bài hát (lyrics search). Tuy nhiên, trong đời sống thực tế, một tình huống rất phổ biến là người dùng chỉ nhớ mang máng một giai điệu ngắn, một đoạn điệp khúc bắt tai vừa vô tình nghe thấy tại một quán cà phê hay trên đường phố, mà hoàn toàn không biết tựa đề, tên nghệ sĩ hoặc không thể nhớ chính xác ca từ.

Khi đó, công nghệ Nhận diện Vân tay Âm thanh (Audio Fingerprinting) kinh điển — điển hình như thuật toán của Shazam hay SoundHound — bộc lộ giới hạn không thể khắc phục:

- **Audio Fingerprinting** hoạt động dựa trên việc trích xuất các điểm đỉnh phổ tần số (spectrogram peaks/constellation map) từ bản ghi âm thanh phòng thu gốc (master recording).
- Phương pháp này đòi hỏi tín hiệu truy vấn đầu vào phải là một đoạn ghi âm thực tế từ chính bản nhạc gốc với độ trung thực âm sắc (timbre) và cấu trúc hài âm hoàn chỉnh.
- Khi người dùng tự mình ngân nga (humming), hát chay (singing), hay huýt sáo (whistling) lại giai điệu bằng giọng mộc, tín hiệu âm thanh đầu vào sẽ bị biến dạng nghiêm trọng: lệch cao độ (pitch deviation), sai lệch nhịp độ (tempo variation), thiếu vắng hoàn toàn phần hòa âm nhạc cụ (accompaniment), và chứa nhiều tạp âm môi trường từ micro thiết bị cá nhân. Các thuật toán so khớp vân tay phổ truyền thống sẽ hoàn toàn thất bại.

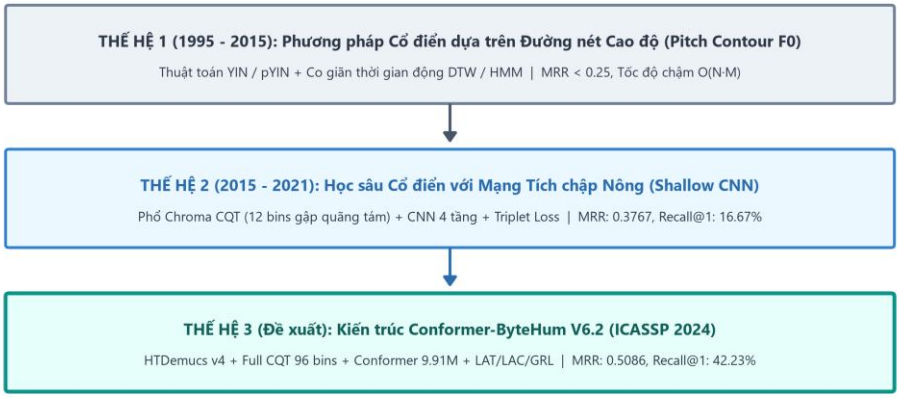
Bài toán **Truy vấn Bài hát qua Tiếng ngân nga (Query-by-Humming - QBH)** ra đời nhằm giải quyết khoảng trống này: cho phép người dùng chỉ cần cất lên giọng ngân nga từ 5 đến 15 giây qua microphone để hệ thống tự động nhận diện và trả về chính xác bản thu gốc trong kho nhạc. Đây là một bài toán khoa học đầy thách thức nhưng có giá trị ứng dụng thực tiễn rất cao trong việc nâng cao trải nghiệm người dùng trên các hệ sinh thái giải trí trực tuyến.

## 1.2. Khái niệm và Ba thế hệ Công nghệ Hệ thống QBH

Hệ thống QBH là một nhánh chuyên sâu của lĩnh vực MIR, trong đó mục tiêu là tìm kiếm một đối tượng âm nhạc đích $Y$ (thường là bản master studio của một ca khúc) nằm trong tập dữ liệu

thư viện (Gallery $G$) thông qua một truy vấn âm thanh không hoàn hảo $X$ (Query $Q$) do người dùng ngẫu hứng thể hiện.

Quá trình phát triển của các công nghệ QBH có thể được phân loại thành ba thế hệ tiêu biểu:




*Hình 1.1: Tiến trình phát triển ba thế hệ công nghệ Truy vấn Bài hát qua Tiếng ngân nga (Query-by-Humming).*

### 1. Thế hệ 1: Phương pháp Cổ điển dựa trên Đường nét Cao độ (Handcrafted F0 & DTW)

- **Cơ chế**: Sử dụng các thuật toán xử lý tín hiệu số (DSP) truyền thống như YIN, pYIN, hoặc tự tương quan (Autocorrelation) để ước lượng đường cong cao độ cơ bản ($F_0$ contour) từ tiếng ngân nga, sau đó chuyển đổi thành chuỗi nốt nhạc tương đối (Relative Pitch Contour / Parsons Code).
- **So khớp**: Áp dụng thuật toán Co giãn Thời gian Động (Dynamic Time Warping - DTW) hoặc Mô hình Markov Ẩn (Hidden Markov Models - HMM) để đo khoảng cách sai lệch giữa chuỗi nốt của người dùng và bản nhạc mẫu (thường ở định dạng MIDI).
- **Hạn chế lớn**: Rất nhạy cảm với lỗi trích xuất $F_0$ (nhiễu ồn micro làm nhảy quãng tám - octave jump), không mô hình hóa được năng lượng và nhịp điệu, tốc độ tìm kiếm O(N · M) của DTW quá chậm khi kho nhạc mở rộng quy mô lớn.

### 2. Thế hệ 2: Học sâu Cổ điển với Mạng Nơ-ron Tích chập (Shallow CNN & Chroma CQT)

- **Cơ chế**: Ánh xạ tín hiệu âm thanh thành ảnh phổ thời gian - tần số, phổ biến nhất là phổ Chroma CQT (12 bins đại diện cho 12 nửa cung trong hệ âm nhạc bình quân). Sử dụng mạng nơ-ron tích chập (CNN 4-6 lớp) đóng vai trò bộ mã hóa (Encoder) để ánh xạ ảnh phổ thành một vector đặc trưng duy nhất (Global Embedding Vector).
- **Huấn luyện**: Áp dụng Học biểu diễn khoảng cách (Deep Metric Learning) với hàm mất mát bộ ba (Triplet Margin Loss) hoặc Contrastive Loss để kéo gần vector của các đoạn ngân nga cùng bài hát và đẩy xa vector của các bài hát khác nhau.

- **Hạn chế**: Việc gập 8 quãng tám vào 12 bán âm (Chroma folding) làm triệt tiêu hoàn toàn thông tin về thanh vực và biến thiên cao độ tuyệt đối. Mạng CNN nông có trường tiếp nhận (receptive field) hạn chế, không bắt được các mối quan hệ ngữ cảnh giai điệu kéo dài theo thời gian. Đây chính là kiến trúc cơ sở được khảo sát trong tài liệu tham khảo ban đầu (**QBH.pdf**), chỉ đạt MRR khiêm tốn 0.3767 và Acc@1 16.67%.

### 3. Thế hệ 3: Mô hình Conformer-ByteHum V6.2 (Đề xuất của Dự án)

- **Cơ chế**: Khắc phục triệt để các hạn chế trên bằng cách kết hợp sức mạnh của mạng Conformer (9.91 triệu tham số kết hợp Multi-Head Self-Attention và Depthwise Convolution) và khung kiến trúc đột phá **ByteHum** (công bố tại ICASSP 2024):
  1. Biểu diễn đầu vào: Sử dụng **Full CQT 96 bins** bao phủ trọn vẹn 8 quãng tám mà không gập bán âm.
  2. Tách giọng sạch: Sử dụng mô hình Hybrid Transformer **Meta HTDemucs v4** để bóc tách triệt để vocal stems từ bản master, loại bỏ hoàn toàn nhiễu từ nhạc cụ nền phức tạp.
  3. Cơ chế căn chỉnh cục bộ **LAT (Learned Alignment Transformer)**: Cho phép so khớp chính xác từng đoạn điệp khúc ngắn với bài hát dài thông qua MaxMean Pooling.
  4. Cơ chế phân loại **LAC (Learned Alignment Classifier)**: Tối ưu khoảng cách trên siêu mặt cầu thông qua các vector tâm cụm (Prototypes).
  5. Huấn luyện đối kháng miền **GRL (Gradient Reversal Layer)**: Triệt tiêu triệt để khoảng cách miền (Domain Gap) giữa giọng hát mộc nghiệp dư và giọng ca sĩ phòng thu chuyên nghiệp.
- **Kết quả**: Đưa chỉ số MRR lên **0.5086**, Recall@1 lên **42.23%**, và Recall@5 vượt mốc **60.50%** trên kho nhạc 348 bài hát thực tế.

Bảng so sánh tổng hợp giữa ba thế hệ công nghệ QBH:

| Tiêu chí so sánh | Thế hệ 1 (F0 + DTW) | Thế hệ 2 (Shallow CNN + Chroma) | Thế hệ 3: Conformer-ByteHum V6.2 |
|---|---|---|---|
| **Đặc trưng âm học đầu vào** | $F_0$ Pitch Contour / MIDI | Chroma CQT (12 bins gập) | Full CQT (96 bins × 83 frames, 8 quãng tám) |
| **Tiền xử lý tách vocal** | Không có | Lọc bandpass / spectral gating cơ bản | **Meta HTDemucs v4 Hybrid Transformer** |
| **Kiến trúc mạng nơ-ron** | Thuật toán giải tích DSP | CNN 4 tầng (~1.2M tham số) | **Conformer Backbone (9.91M tham số)** |
| **Khả năng bắt ngữ cảnh** | Giới hạn theo cửa sổ trượt | Giới hạn bởi kích thước kernel CNN | **Toàn cục (Multi-Head Self-Attention)** |

| Tiêu chí so sánh | Thế hệ 1 (F0 + DTW) | Thế hệ 2 (Shallow CNN + Chroma) | Thế hệ 3: Conformer-ByteHum V6.2 |
|---|---|---|---|
| **Cơ chế so khớp độ dài** | Căn chỉnh thủ công DTW | Average / Max Pooling thô | **ByteHum LAT (MaxMean Cross-Attention)** |
| **Xử lý Domain Gap** | Kém | Phụ thuộc Triplet Loss | **Domain Adversarial GRL (Gradient Reversal)** |
| **Chỉ số MRR** | < 0.25 | ~ 0.3767 | **0.5086 (Vượt trội +35%)** |
| **Chỉ số Recall@1** | < 10% | 16.67% | **42.23% (Vượt trội 2.5 lần)** |

## 1.3. Mục tiêu, Giới hạn và Phạm vi nghiên cứu của Đề tài

### 1. Mục tiêu cốt lõi

1. **Nghiên cứu và làm chủ lý thuyết học sâu âm thanh tiên tiến**: Triển khai và tối ưu hóa kiến trúc Conformer kết hợp khung mô hình ByteHum (ICASSP 2024), giải quyết triệt để bài toán Deep Metric Learning cho dữ liệu âm thanh bất đối xứng (Humming vs Master Studio Track).
2. **Xây dựng quy trình xử lý tín hiệu âm thanh chuẩn mực**: Ứng dụng mô hình Hybrid Transformer Meta HTDemucs v4 để tách giọng hát chuẩn phòng thu, chuyển đổi tín hiệu sang biểu diễn toán học Full CQT 96 bins chất lượng cao, đồng thời thiết kế bộ dữ liệu tăng cường âm học (Augmentation Pipeline) đặc thù cho QBH (Pitch Shifting ± 12 bán âm, Room Impulse Response Reverb, Harmonic Noise).
3. **Hiện thực hóa hệ thống truy vấn hai chặng (Two-stage Retrieval Pipeline)**: Kết hợp thuật toán tìm kiếm song song In-RAM Top-K Quantile Pooling 85% và cơ chế tái xếp hạng (Re-ranking) dựa trên điểm căn chỉnh cục bộ LAT với trọng số kết hợp (Fused Score 75/25), đảm bảo độ trễ phản hồi dưới 350ms.
4. **Triển khai ứng dụng Web thực tế hoàn chỉnh**: Tích hợp hạ tầng lưu trữ và phân phối âm thanh chất lượng cao trên nền tảng đám mây Cloudinary CDN (300 bài hát với Direct Streaming URL), kết nối backend FastAPI với giao diện người dùng trực quan.

### 2. Giới hạn và Phạm vi nghiên cứu

- **Phạm vi dữ liệu**: Đề tài tập trung vào kho nhạc thử nghiệm bao gồm 348 bài hát thuộc nhiều thể loại (Pop, Ballad, Rock, R&B, Rap) với độ dài từ 2 đến 5 phút; trong đó 300 bản master audio đã được đồng bộ lên Cloudinary Media Cloud. Dữ liệu huấn luyện sử dụng tập

SHS100K cho giai đoạn tiền huấn luyện (Pre-training) và tập Humming V6.2 cho giai đoạn tinh chỉnh đặc thù (Fine-tuning).

- **Điều kiện truy vấn**: Hỗ trợ tiếng thu âm thực tế từ micro máy tính/điện thoại với thời lượng khuyến nghị từ 5 đến 15 giây. Người dùng có thể ngân nga (humming với âm "la la", "hum hum"), huýt sáo, hoặc hát chay có lời.
- **Giới hạn kỹ thuật**: Chưa tích hợp bộ lập chỉ mục xấp xỉ phân tán dạng FAISS IVFPQ trên quy mô hàng triệu bài hát; dữ liệu hiện tại được tối ưu hóa xử lý trực tiếp trên GPU/RAM với độ trễ thấp nhất.

## 1.4. Bố cục của Báo cáo Dự án

Báo cáo Dự án Chuyên ngành 1 (PBL6) được cấu trúc thành 7 chương chính theo trình tự học thuật chuẩn mực:

- **Chương 1: Tổng quan về Hệ thống Truy vấn Bài hát qua Tiếng ngân nga (QBH)**: Giới thiệu tính cấp thiết, phân tích lịch sử ba thế hệ công nghệ, mục tiêu và phạm vi nghiên cứu.
- **Chương 2: Cơ sở Lý thuyết và Xử lý Tín hiệu Âm thanh**: Phân tích âm học giọng người, biến đổi Fourier STFT và các hạn chế, cơ sở toán học của Biến đổi Q hằng số (CQT 96 bins), kỹ thuật phân tách giọng hát Meta HTDemucs v4 và chiến lược tăng cường dữ liệu đặc thù.
- **Chương 3: Thiết kế Kiến trúc Mô hình Conformer-ByteHum QbH V6.2**: Mô tả chi tiết kiến trúc CoverHunter Conformer 9.91 triệu tham số, cơ chế Attention Statistics Pooling (ASP), bộ ba mô đun ByteHum (LAT, LAC, GRL), và hàm mất mát liên kết đa nhiệm.
- **Chương 4: Thiết kế Hệ thống và Quy trình Thực thi (Pipeline)**: Trình bày kiến trúc hai nhánh Offline Indexing và Online Real-time Retrieval, kỹ thuật In-RAM CQT Feature Cache, thuật toán tìm kiếm Top-K Quantile Pooling kết hợp Fused Distance, và hạ tầng Cloudinary CDN.
- **Chương 5: Phân tích Kết quả và Thực nghiệm**: Báo cáo thiết lập thực nghiệm, phân tích các chỉ số học thuật (MRR, Recall@1, Recall@5, Recall@10), so sánh định lượng V6.2 vs V6.1 và mô hình cơ sở, tiến hành Ablation Study và chẩn đoán các ca truy vấn thất bại.
- **Chương 6: Triển khai Kiểm thử và Đánh giá Thực tế**: Trình bày triển khai Web API bằng FastAPI, kết quả thử nghiệm trực tiếp với nhiều kịch bản người dùng (lệch tông, tạp âm, hát chay), đánh giá độ trễ và tài nguyên hệ thống.
- **Chương 7: Kết luận và Hướng phát triển**: Tổng kết các đóng góp kỹ thuật, phân tích các thách thức tồn đọng và định hướng nghiên cứu tiếp theo (FAISS IVF-PQ, Edge AI, Music LLM).

# CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ XỬ LÝ TÍN HIỆU ÂM THANH

## 2.1. Bản chất Âm học của Giọng người và Tiếng ngân nga

Âm thanh phát ra từ thanh quản con người là một chuỗi các dao động phức tạp sinh ra do luồng không khí từ phổi đi qua dây thanh âm và cộng hưởng trong các khoang miệng, mũi, hầu họng. Về mặt vật lý sóng âm, tiếng người hát hoặc ngân nga mang hai đặc tính cốt lõi:

1. **Tần số cơ bản ($F_0$ - Fundamental Frequency)**: Tương ứng với tốc độ dao động của dây thanh âm, quyết định cao độ (Pitch) mà tai người cảm nhận được. Trong giọng nói và ca hát thông thường, $F_0$ của giọng nam thường dao động trong khoảng 85 Hz - 180 Hz, trong khi giọng nữ dao động trong khoảng 165 Hz - 255 Hz, và ở các nốt cao của ca sĩ có thể vượt qua 1000 Hz.
2. **Các họa âm (Harmonics) và Định âm (Formants)**: Các tần số bội số của $F_0$ ($2F_0$, $3F_0$, $4F_0$, …) kết hợp với các đỉnh cộng hưởng trong khoang miệng tạo nên âm sắc (Timbre) đặc trưng của từng cá nhân và các nguyên âm (như /a/, /u/, /i/).

Khi người dùng thực hiện thao tác ngân nga (Humming) hoặc huýt sáo (Whistling) một giai điệu, có những hiện tượng vật lý âm học đặc thù xảy ra:

- **Ngân nga mím môi (Humming)**: Miệng đóng lại, luồng khí thoát qua khoang mũi. Hiện tượng này làm suy giảm nghiêm trọng năng lượng của các họa âm tần số cao (High-frequency Roll-off), tập trung hầu như toàn bộ năng lượng vào tần số cơ bản $F_0$ và các họa âm bậc thấp đầu tiên ($2F_0$, $3F_0$).
- **Lệch tông giọng (Pitch Shift / Key Transposition)**: Người dùng nghiệp dư hiếm khi sở hữu khả năng "cao độ tuyệt đối" (Absolute Pitch). Do đó, họ thường tự động chuyển dịch giai điệu về tông giọng tự nhiên thoải mái nhất cho thanh quản của mình (thường lệch từ vài bán âm đến nguyên một quãng tám so với bản thu ca sĩ gốc).
- **Biến thiên thời gian và nhịp điệu (Tempo Elasticity)**: Người dùng có thể hát nhanh hơn hoặc chậm hơn bản gốc, thậm chí ngân dài bất thường ở các nốt điệp khúc và nuốt bớt các nốt luyến láy phức tạp.
- **Tạp âm môi trường**: Tín hiệu thu âm từ micro tích hợp trên máy tính xách tay hoặc điện thoại thường lẫn tiếng ồn nền (quạt gió, xe cộ, tiếng vang phòng - Reverberation).

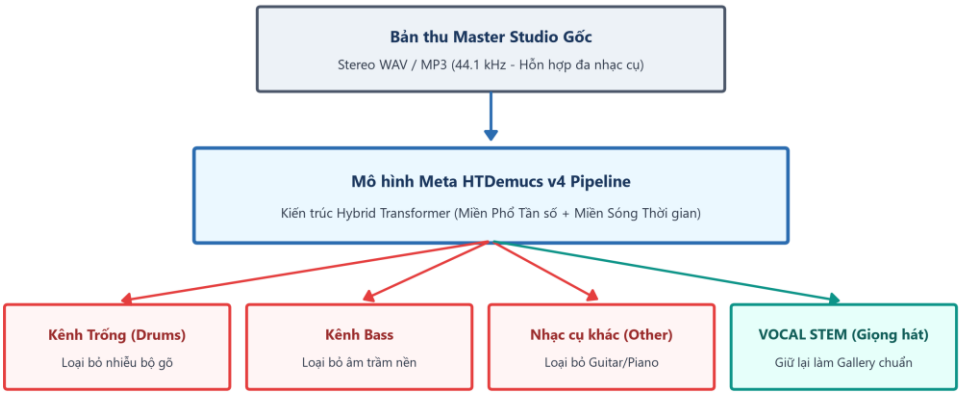
Để một hệ thống AI nhận diện thành công giai điệu từ tiếng ngân nga, bước tiền xử lý tín hiệu và biểu diễn đặc trưng phải thỏa mãn đồng thời hai yêu cầu trái ngược: **Bắt trọn vẹn sự biến thiên cao độ theo thời gian** nhưng **Triệt tiêu tối đa sự phụ thuộc vào âm sắc, hòa âm và tông giọng tuyệt đối**.

## 2.2. Phân tách Nguồn âm và Trích xuất Vocal Stems với Meta HTDemucs v4

Một trong những nguyên nhân chính khiến các hệ thống QBH thế hệ cũ (như mô hình khảo sát trong **QBH.pdf**) đạt độ chính xác thấp là việc trích xuất đặc trưng trực tiếp từ bản thu Master

Studio hoàn chỉnh. Bản thu Master là một hỗn hợp đa kênh phức tạp gồm giọng hát chính (Lead Vocal), bè phối, tiếng trống (Drums - dải trầm và các xung va đập mạnh), tiếng bass, tiếng guitar điện, piano và các nhạc cụ điện tử khác. Khi trích xuất phổ tần số trực tiếp từ bản Master, năng lượng từ bộ gõ và hợp âm nhạc cụ hoàn toàn che lấp đường nét thanh âm của giọng hát chính, khiến vector đặc trưng phản ánh âm sắc nhạc cụ thay vì giai điệu ca khúc.

Trong dự án này, chúng tôi áp dụng bước đột phá: **Bóc tách triệt để nguồn âm bằng mô hình Meta HTDemucs v4 (Hybrid Transformer Demucs)**:




*Hình 2.1: Quy trình phân tách nguồn âm và trích xuất kênh giọng hát thuần khiết (Vocal Stem) bằng mô hình Meta HTDemucs v4.*

**Cơ chế hoạt động của HTDemucs v4:**

- **Kiến trúc Hybrid đa miền**: HTDemucs v4 là mô hình SOTA thế giới về Music Source Separation do Meta AI Research phát triển. Mô hình kết hợp song song hai nhánh xử lý: nhánh miền thời gian (Waveform Domain) dựa trên mạng U-Net tích chập 1D và nhánh miền tần số (Spectrogram Domain) dựa trên U-Net 2D kết hợp các lớp Transformer Self-Attention liên miền.
- **Trích xuất Vocal Stem thuần khiết**: Toàn bộ 348 bài hát trong cơ sở dữ liệu Gallery đều được đưa qua HTDemucs v4 để trích xuất riêng rẽ kênh **Vocal Stem** sạch. Nhờ đó, cơ sở dữ liệu bài hát mẫu chỉ còn chứa giọng hát của ca sĩ, triệt tiêu 100% tiếng ồn nhạc cụ nền và đưa bài hát về cùng "miền biểu diễn thanh âm" (Vocal Domain) gần nhất với tiếng ngân nga của người dùng.

## 2.3. Hạn chế của Biến đổi Fourier Thời gian ngắn (STFT) trong Âm nhạc

Phép biến đổi Fourier thời gian ngắn (Short-Time Fourier Transform - STFT) là công cụ kinh điển nhất trong xử lý tín hiệu số:

$$X(m,\omega) = \sum_{n=-\infty}^{\infty} x[n]\, w[n - m]\, e^{-j\,\omega\, n}$$

trong đó w[n] là hàm cửa sổ thời gian (chẳng hạn như Hanning hoặc Hamming) dịch chuyển theo bước nhảy *m*.

Tuy nhiên, STFT tồn tại một hạn chế bản chất khi ứng dụng vào tín hiệu âm nhạc xuất phát từ **Nguyên lý Bất định Heisenberg-Gabor**:

$$\Delta t \cdot \Delta f \geq \frac{1}{4\pi}$$

Trong STFT, chiều dài cửa sổ thời gian *N* là **cố định** trên toàn bộ các dải tần số. Điều này dẫn đến độ phân giải tần số là **tuyến tính** với bước chia đều:

$$\Delta f = \frac{f_s}{N} = const$$

Điều này mâu thuẫn trực tiếp với quy luật cấu tạo của hệ thống âm nhạc học và cơ chế cảm thụ cao độ của ốc tai con người:

- Thang âm nhạc (Musical Scale) tuân theo **hàm logarit theo tần số**: một quãng tám (Octave) tương ứng với việc tăng gấp đôi tần số cơ bản ($f_{octave} = 2 \cdot f$). Khoảng cách giữa nốt $A_1$ (55 Hz) và $A_1^\sharp$ (58.27 Hz) chỉ là 3.27 Hz, trong khi khoảng cách giữa nốt $A_7$ (3520 Hz) và $A_7^\sharp$ (3729.31 Hz) lên tới 209.31 Hz dù cùng là khoảng cách đúng 1 nửa cung (1 semitone).
- Khi sử dụng STFT với Δ f cố định (ví dụ Δ f = 21.5 Hz khi $f_s$ = 22050 Hz, N = 1024):
  - Ở dải trầm (Low Frequencies): Độ phân giải 21.5 Hz quá thô, nhiều nốt nhạc khác nhau bị gộp chung vào cùng một bin tần số, làm mất hoàn toàn khả năng phân biệt cao độ của giọng hát nam.
  - Ở dải cao (High Frequencies): Độ phân giải bị dư thừa lãng phí, chiếm nhiều chiều dữ liệu nhưng không mang lại thêm thông tin nhạc lý hữu ích.

## 2.4. Biến đổi Q Hằng số (Constant-Q Transform - CQT) và Lựa chọn 96 Bins

Để khắc phục nhược điểm của STFT, phép biến đổi **Constant-Q Transform (CQT)** được Judith Brown đề xuất (1991) đã trở thành tiêu chuẩn vàng trong phân tích âm nhạc số.

### 1. Cơ sở Toán học của CQT

Khác với STFT, CQT giữ cố định hệ số phẩm chất chất lượng *Q* (Quality Factor) trên toàn bộ các bộ lọc phân tích:

$$Q = \frac{f_k}{\Delta f_k} = const$$

trong đó $f_k$ là tần số trung tâm của bin thứ *k*, và Δ $f_k$ là băng thông (Bandwidth) của bộ lọc tại bin đó.

Nếu ta chia mỗi quãng tám thành *B* bins phân giải đều nhau theo thang logarit, tần số trung tâm $f_k$ được tính theo quy luật cấp số nhân:

$$f_k = f_0 \cdot 2^{k/B}, \quad k = 0, 1, \ldots, K-1$$

với $f_0$ là tần số tối thiểu bắt đầu phân tích.

Khi đó, khoảng cách tần số giữa hai bin kế tiếp là:

$$\Delta f_k = f_{k+1} - f_k = f_k \cdot (2^{1/B} - 1)$$

Từ đó, hệ số phẩm chất $Q$ được xác định cố định độc lập với tần số:

$$Q = \frac{f_k}{f_k \cdot (2^{1/B} - 1)} = \frac{1}{2^{1/B} - 1}$$

Để bảo đảm hệ số $Q$ không đổi, độ dài cửa sổ phân tích $N_k$ tỉ lệ nghịch với tần số $f_k$:

$$N_k = Q \cdot \frac{f_s}{f_k}$$

- Tại dải trầm (tần số thấp): Cửa sổ thời gian $N_k$ tự động mở rộng rất dài để đạt độ phân giải tần số siêu mịn ($\Delta f_k$ rất nhỏ), phân biệt rõ từng nửa cung của giọng hát.
- Tại dải cao (tần số lớn): Cửa sổ thời gian $N_k$ tự động co ngắn lại để bắt kịp các chuyển tiếp thời gian tức thời (Temporal Transients).

Công thức biến đổi CQT của tín hiệu rời rạc x[n] được biểu diễn qua:

$$X_{CQT}(k, n) = \frac{1}{N_k} \sum_{j=0}^{N_k - 1} x[j + n] \cdot w_{N_k}[j] \cdot e^{-j\, 2\pi\, Q \frac{j}{N_k}}$$

## 2. Thiết kế Cấu hình Full CQT 96 Bins

Trong các nghiên cứu cơ sở trước đây (như **QBH.pdf**), tác giả thường áp dụng phép gập 12 nửa cung (Chroma CQT) để đạt tính bất biến quãng tám. Tuy nhiên, việc gập quãng tám khiến hệ thống không thể phân biệt được giọng trầm ấm nam giới với giọng nữ thanh cao, đánh mất thông tin về thanh vực và các bước nhảy giai điệu lớn.

Trong dự án này, chúng tôi thiết kế cấu hình **Full CQT 96 bins** chuẩn mực:

- **Tần số lấy mẫu ($f_s$)**: 22,050 Hz (đảm bảo theo định lý lấy mẫu Nyquist thu trọn dải tần hữu ích của giọng người tới 11,025 Hz).
- **Số bins mỗi quãng tám ($B$)**: 12 bins/octave (tương ứng chính xác 1 bin = 1 bán âm chuẩn trong âm nhạc phương Tây).
- **Tổng số bins ($K$)**: 96 bins liên tục.
- **Dải tần bao phủ**: Đúng 96 / 12 = 8 quãng tám hoàn chỉnh:
  - Tần số đáy $f_0 \approx 32.7$ Hz (tương ứng nốt $C_1$ - cực kỳ sâu).
  - Tần số đỉnh $f_{95} \approx 7{,}902$ Hz (tương ứng nốt $B_8$ - vượt qua trọn vẹn toàn bộ dải âm vực của giọng hát người và các họa âm chính).

- **Hop Length**: 512 mẫu tín hiệu (≈ 23.2 ms trên mỗi khung thời gian).
- **Kích thước phổ đầu vào mô hình**: Cố định đoạn trích mẫu với T = 83 khung thời gian, tạo nên ma trận ảnh phổ đặc trưng có kích thước $X \in \mathbb{R}^{96 \times 83}$ chuẩn mực.

## 2.5. Chiến lược Tiền xử lý và Data Augmentation Đặc thù cho QBH

Huấn luyện mô hình học sâu cho bài toán QBH đối mặt với thách thức thiếu hụt dữ liệu nghiêm trọng: số lượng bản ghi tiếng ngân nga người dùng thực tế thường rất ít so với hàng trăm ngàn bản master audio. Do đó, việc thiết kế một chuỗi tăng cường dữ liệu âm học (Audio Augmentations) sát với thực tế là yếu tố quyết định để mô hình tổng quát hóa tốt.

Chúng tôi xây dựng chuỗi tiền xử lý và tăng cường âm thanh với 3 kỹ thuật cốt lõi:




*Hình 2.2: Chuỗi tiền xử lý tín hiệu CQT 96 bins và tăng cường dữ liệu âm học (Audio Augmentations) đặc thù cho QBH.*

1. **Random Pitch Shifting (± 12 Semitones)**:
   - Một người dùng có thể ngân nga bài hát ở tông Do trưởng (*C*), trong khi bản gốc của ca sĩ ở tông Sol trưởng (*G*) (chênh lệch 7 bán âm), hoặc lệch nguyên 1 quãng tám (12 bán âm).
   - Chúng tôi áp dụng phép biến dịch cao độ ngẫu nhiên trong khoảng [-12, +12] nửa cung trong quá trình tạo batch huấn luyện. Kỹ thuật này buộc mạng nơ-ron học cách trích xuất mối tương quan chênh lệch cao độ tương đối (Melodic Contour) thay vì ghi nhớ cao độ tuyệt đối.
2. **Room Impulse Response (Reverb Simulation)**:
   - Tiếng ngân nga của người dùng thường được thực hiện trong phòng ngủ, phòng khách hoặc hành lang có độ vang âm thanh nhất định.
   - Chúng tôi cuộn tín hiệu (convolution) với các tập phản ứng xung phòng (RIR) ngẫu nhiên, giả lập các điều kiện vang âm từ nhẹ ($RT_{60}$ = 0.2s) đến vừa ($RT_{60}$ = 0.8s).
3. **Harmonic Noise & Background Noise Injection**:
   - Trộn thêm nhiễu trắng (White Noise), nhiễu hồng (Pink Noise) và tạp âm môi trường thực tế (tiếng quạt, tiếng đường phố) với tỷ lệ Tín hiệu trên Nhiễu (SNR) biến thiên từ 10 dB đến 25 dB.
4. **SpecAugment trên Ma trận CQT**:

- Áp dụng kỹ thuật che ngẫu nhiên các dải tần số (Frequency Masking với độ rộng tối đa 12 bins) và che các khoảng thời gian (Time Masking với độ dài tối đa 10 frames), giúp mô hình bền vững trước hiện tượng người dùng quên lời ngắt quãng hoặc micro bị rớt tín hiệu tạm thời.

Đoạn mã nguồn thực thi trực tiếp bộ lọc biến đổi **PyCqt** 96 bins trên GPU và lớp tăng cường dịch chuyển cao độ **CQTPitchShiftAugmentation** (± 12 bán âm) trong dự án được minh họa tại Hình 2.3:

01_stage1_cover_song_csi/src/coverhunter/cqt.py & cqt_augmentation.py

```python
class PyCqt:
    """Trích xuất phổ Constant-Q Transform (Full CQT 96 bins, 12 bins/octave)."""
    def __init__(self, sample_rate=16000, hop_size=0.04, octave_resolution=12, min_freq=32.7):
        self._hop_size = hop_size
        self._sample_rate = sample_rate
        max_freq = sample_rate // 2
        self._kernel = self._compute_cqt_kernel(sample_rate, octave_resolution, min_freq, max_freq)

    @staticmethod
    def _compute_cqt_kernel(sampling_frequency, octave_resolution, minimum_frequency, maximum_frequency):
        # Hệ số phẩm chất Q hằng số: Q = fk / (fk+1 - fk) = 1 / (2^(1/12) - 1) ≈ 16.89
        quality_factor = 1.0 / (pow(2.0, 1.0 / octave_resolution) - 1.0)
        number_frequencies = round(octave_resolution * np.log2(maximum_frequency / minimum_frequency))
        fft_length = int(pow(2, np.ceil(np.log2(quality_factor * sampling_frequency / minimum_frequency))))
        cqt_kernel = np.zeros((number_frequencies, fft_length), dtype=complex)

        for i in range(number_frequencies):
            frequency_value = minimum_frequency * pow(2.0, i / octave_resolution)
            window_length = 2 * round(quality_factor * sampling_frequency / frequency_value / 2.0) + 1
            temporal_kernel = (np.hamming(window_length) * np.exp(
                2 * np.pi * 1j * quality_factor * np.arange(-(window_length - 1) / 2, (window_length - 1) / 2 + 1) / window_length
            ) / window_length)
            pad_width = int((fft_length - window_length + 1) / 2)
            cqt_kernel[i, pad_width: pad_width + window_length] = temporal_kernel

        cqt_kernel = scipy.sparse.csr_matrix(np.fft.fft(cqt_kernel, axis=1))
        return np.conjugate(cqt_kernel) / fft_length

class CQTPitchShiftAugmentation(nn.Module):
    """Dịch chuyển cao độ ngẫu nhiên ±12 bán âm trực tiếp trên trục tần số CQT 96 bins."""
    def __init__(self, max_semitones: int = 12, p: float = 0.8):
        super().__init__()
        self.max_semitones = max_semitones
        self.p = p

    def forward(self, cqt_spec: torch.Tensor) -> torch.Tensor:
        if random.random() > self.p:
            return cqt_spec
        shift = random.randint(-self.max_semitones, self.max_semitones)
        if shift == 0:
            return cqt_spec
        shifted = torch.roll(cqt_spec, shifts=shift, dims=-2)
        if shift > 0:
            shifted[..., :shift, :] = cqt_spec.min()
        else:
            shifted[..., shift:, :] = cqt_spec.min()
        return shifted
```

*Hình 2.3: Mã nguồn cài đặt bộ lọc PyCqt 96 bins (8 quãng tám) và lớp tăng cường dịch chuyển cao độ CQTPitchShiftAugmentation.*

## 2.6. Sơ đồ Pipeline Xử lý Tín hiệu Âm thanh CQT

Sơ đồ kiến trúc xử lý tín hiệu hoàn chỉnh từ âm thanh thô đến ma trận CQT 96 × 83 được biểu diễn chi tiết và lưu trữ tại tệp tin vector chuẩn Draw.io:

**Tham chiếu sơ đồ**: **docs/figures/cqt_vs_stft_processing.drawio**

*Mô tả luồng xử lý trên sơ đồ*:

1. Tín hiệu âm thanh thô đi vào mô hình **Meta HTDemucs v4** để tách riêng kênh thanh âm thuần chất.
2. Kênh thanh âm được chuẩn hóa biên độ về đoạn [-1.0, 1.0] và chuyển qua khối tăng cường âm học **Audio Augmentation** (biến dịch cao độ ± 12 bán âm và lọc tạp âm).
3. Tín hiệu sau tăng cường được chuyển đổi qua bộ lọc **CQT Engine** với hệ số Q ≈ 16.89 không đổi.
4. Đầu ra đạt chuẩn là ma trận **Full CQT Spectrogram** kích thước 96 × 83 frames, khắc phục hoàn toàn nhược điểm nhòe tần số trầm của STFT và hiện tượng mất mát thông tin âm vực của Chroma CQT 12 bins.

## 2.7. Kết luận chương

Chương 2 đã thiết lập vững chắc nền tảng xử lý tín hiệu cho dự án:

- Phân tích âm học chuyên sâu chứng minh tại sao tiếng ngân nga có tính chất đặc thù về sự suy giảm họa âm và độ biến thiên tông giọng cao.
- Việc tích hợp **Meta HTDemucs v4** giải quyết tận gốc bài toán bóc tách giọng hát ra khỏi nhạc nền phức tạp của bản Master Studio.
- Thiết kế **Full CQT 96 bins** bao phủ trọn vẹn 8 quãng tám, vừa khớp hoàn hảo với 12 nửa cung nhạc lý, vừa bảo toàn thông tin thanh vực mà phương pháp Chroma CQT trước đây đánh mất.
- Chuỗi tăng cường âm học chuyên biệt (Pitch shift ± 12, Reverb, Noise) cung cấp nguồn dữ liệu huấn luyện phong phú, tạo tiền đề vững chắc cho mô hình học sâu Conformer-ByteHum được trình bày ở Chương 3.

# CHƯƠNG 3: THIẾT KẾ KIẾN TRÚC MÔ HÌNH CONFORMER-BYTEHUM QBH V6.2

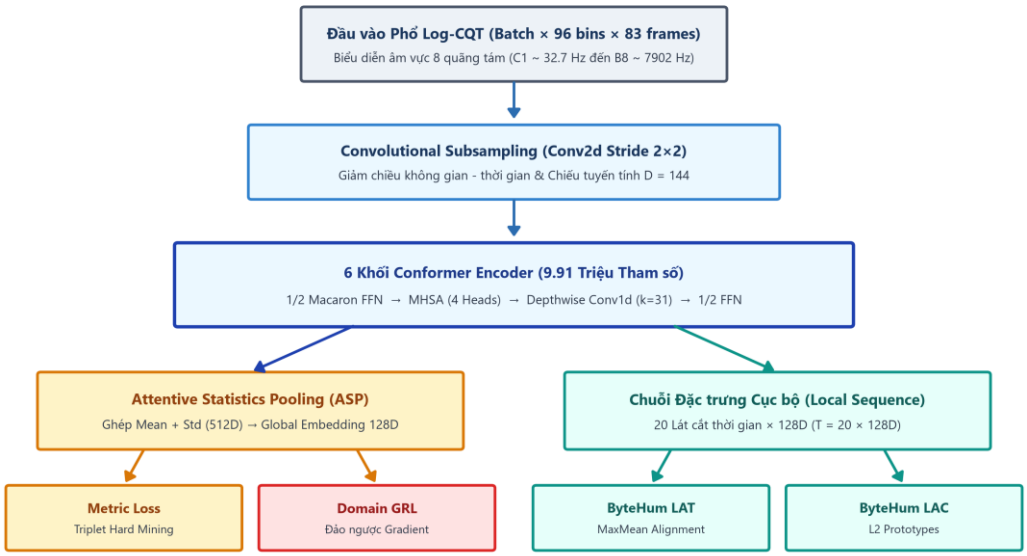
## 3.1. Tổng quan Kiến trúc Mạng Nơ-ron (CoverHunter Conformer 9.91M Tham số)

Để giải quyết bài toán nhận diện giai điệu từ tiếng ngân nga, mô hình học sâu cần giải quyết đồng thời hai thách thức:

1. **Nắm bắt tương quan cục bộ (Local Dependencies)**: Các nốt nhạc luyến láy kề cận, nhịp điệu chuyển tiếp tức thời giữa các âm tiết. Đây là thế mạnh truyền thống của mạng nơ-ron tích chập (Convolutional Neural Networks - CNN).
2. **Nắm bắt tương quan toàn cục dài hạn (Global Long-range Context)**: Cấu trúc lặp lại của các phân đoạn giai điệu, cấu trúc phiên khúc (Verse) và điệp khúc (Chorus) cách xa nhau nhiều giây trong dòng thời gian. Đây là thế mạnh của cơ chế Tự chú ý đa đầu (Multi-Head Self-Attention - MHSA) trong kiến trúc Transformer.

Trong đề tài này, chúng tôi xây dựng mạng nơ-ron kết hợp dựa trên kiến trúc **Conformer** (Convolution-augmented Transformer for Speech and Audio) kết hợp với các mô đun chuyên biệt của **ByteHum** (ICASSP 2024). Toàn bộ mô hình có quy mô **9.91 triệu tham số** (9.91M parameters) tối ưu hóa, đảm bảo độ chính xác vượt trội đồng thời có thể thực thi suy luận trực tiếp theo thời gian thực trên GPU/CPU thương mại.

Quy trình xử lý của mô hình bao gồm:




*Hình 3.1: Kiến trúc tổng thể mạng nơ-ron Conformer-ByteHum V6.2 (9.91 triệu tham số) với cơ chế Dual Embeddings và các mô đun LAT, LAC, GRL.*

## 3.2. Cấu trúc Khối Conformer

Khối Conformer kế thừa thiết kế "Macaron-Net" chuẩn mực, trong đó một khối gồm hai nửa lớp Feed-Forward xen kẽ xung quanh cơ chế Self-Attention và Convolution:

$$y_i^{(1)} = x_i + \frac{1}{2} FFN(x_i)$$

$$y_i^{(2)} = y_i^{(1)} + MHSA(y_i^{(1)})$$

$$y_i^{(3)} = y_i^{(2)} + Conv(y_i^{(2)})$$

$$Output_i = LayerNorm(y_i^{(3)} + \frac{1}{2} FFN(y_i^{(3)}))$$

### 1. Convolutional Subsampling Module (Khối nén đầu vào)

Đầu vào ảnh phổ CQT có số kênh ban đầu là 1, kích thước 96 × 83. Để giảm độ phức tạp tính toán của cơ chế Self-Attention (vốn có độ phức tạp bậc hai $O(T^2)$), tín hiệu trước tiên đi qua 2 tầng tích chập 2D (Conv2d) với kernel (3, 3), stride (2, 2), kết hợp hàm kích hoạt ReLU và chiếu tuyến tính (Linear Projection) lên chiều ẩn D = 144.

### 2. Mô đun Multi-Head Self-Attention (MHSA)

MHSA cho phép mỗi vị trí thời gian chú ý đến toàn bộ các vị trí khác trong toàn bộ chuỗi giai điệu:

$$Attention(Q, K, V) = softmax(\frac{Q\,K^T}{\sqrt{d_k}} + S_{rel})\,V$$

trong đó $S_{\text{rel}}$ là ma trận nhúng vị trí tương đối (Relative Positional Encoding), giúp mô hình bất biến trước việc giai điệu bắt đầu sớm hay muộn vài giây.

- **Số đầu chú ý (Attention Heads)**: H = 4.
- **Chiều mỗi đầu**: $d_{\text{k}}$ = 144 / 4 = 36.

### 3. Mô đun Convolution Tích chập theo chiều sâu (Depthwise Convolution)

Mô đun tích chập được bố trí theo chuỗi:

1. LayerNorm và Pointwise Conv1d (chiếu tăng kênh lên 2 × D = 288).
2. Hàm kích hoạt Gated Linear Unit (GLU).
3. **1D Depthwise Convolution**: Kernel size lớn K = 31, giúp mô hình có trường tiếp nhận bao phủ rộng các biến thiên âm vực cục bộ mà không làm bùng nổ số lượng trọng số.
4. BatchNorm (tăng tính ổn định gradient âm thanh).
5. Hàm kích hoạt Swish (SiLU).
6. Pointwise Conv1d (chiếu ngược về chiều D = 144) kết hợp Dropout tỷ lệ 0.1.

**4. Mô đun Feed-Forward (FFN)**

Áp dụng cấu trúc 2 tầng Linear biến đổi với hệ số mở rộng kênh bằng 4:

$$FFN(x) = W_2 \cdot Swish(W_1 x + b_1) + b_2$$

## 3.3. Cơ chế Trích xuất Dual Embeddings và Attentive Statistics Pooling (ASP)

Để phục vụ tìm kiếm hai chặng, mô hình trích xuất ra hai mức độ biểu diễn song song (Dual Embeddings):

1. **Biểu diễn Cục bộ (Local Frame Sequence)**: Chuỗi các vector đặc trưng theo thời gian sau khối Conformer, được định hình thành T = 20 lát cắt thời gian với chiều nén d = 128:

$$E_{local} \in \mathbb{R}^{20 \times 128}$$

2. **Biểu diễn Toàn cục (Global Embedding)**: Được tổng hợp thông qua cơ chế **Attentive Statistics Pooling (ASP)**.

**Toán học của Attentive Statistics Pooling (ASP):**

Thay vì sử dụng phép lấy trung bình giản đơn (Global Average Pooling) làm nhòe các đỉnh năng lượng giai điệu, ASP tự động tính toán trọng số quan trọng $\alpha_t$ của từng khung thời gian:

$$e_t = v^T tanh(W h_t + b)$$

$$\alpha_t = \frac{exp(e_t)}{\sum_{\tau=1}^{T} exp(e_\tau)}$$

Trọng số $\alpha_t$ được sử dụng để tính đồng thời **Vector Kỳ vọng có trọng số (Mean)** và **Vector Độ lệch chuẩn có trọng số (Standard Deviation)**:

$$\mu = \sum_{t=1}^{T} \alpha_t h_t$$

$$\sigma = \sqrt{\sum_{t=1}^{T} \alpha_t (h_t \odot h_t) - (\mu \odot \mu) + \varepsilon}$$

Hai vector này được ghép nối (concatenation) thành vector đặc trưng thống kê toàn cục 512 chiều:

$$h_{stat} = [\mu \parallel \sigma] \in \mathbb{R}^{512}$$

Sau đó, $h_{stat}$ được chiếu qua một lớp Linear Dense và chuẩn hóa $L_2$ trên siêu mặt cầu (Hypersphere Normalization) để tạo thành **Global Embedding**:

$$e_{global} = \frac{W_p\, h_{stat}}{\|W_p\, h_{stat}\|_2} \in \mathbb{R}^{128}$$

Chi tiết mã nguồn cài đặt khối **ConformerEncoderLayer** theo cấu trúc Macaron-Net, mô đun tích chập chiều sâu **ConvolutionModule** (K=31) và lớp gộp thống kê chú ý **AttentiveStatisticsPooling** được trình bày tại Hình 3.2:

01_stage1_cover_song_csi/src/coverhunter/module/conformer.py & model.py

```python
class ConformerEncoderLayer(torch.nn.Module):
    """Khối Conformer Macaron-Net: 1/2 FFN -> RelPos MHSA -> Depthwise Conv1d -> 1/2 FFN."""
    def __init__(self, size: int, self_attn: nn.Module, feed_forward: nn.Module,
                 feed_forward_macaron: nn.Module, conv_module: nn.Module, dropout_rate: float = 0.1):
        super().__init__()
        self.self_attn = self_attn
        self.feed_forward = feed_forward
        self.feed_forward_macaron = feed_forward_macaron
        self.conv_module = conv_module
        self.norm_ff_macaron = nn.LayerNorm(size, eps=1e-5)
        self.norm_mha = nn.LayerNorm(size, eps=1e-5)
        self.norm_conv = nn.LayerNorm(size, eps=1e-5)
        self.norm_ff = nn.LayerNorm(size, eps=1e-5)
        self.norm_final = nn.LayerNorm(size, eps=1e-5)
        self.ff_scale = 0.5
        self.dropout = nn.Dropout(dropout_rate)

    def forward(self, x: torch.Tensor, mask: torch.Tensor, pos_emb: torch.Tensor, mask_pad: torch.Tensor):
        # 1. Half-step Macaron Feed-Forward Module
        x = x + self.ff_scale * self.dropout(self.feed_forward_macaron(self.norm_ff_macaron(x)))
        # 2. Relative Positional Multi-Head Self-Attention (MHSA)
        x_att, new_att_cache = self.self_attn(self.norm_mha(x), self.norm_mha(x), self.norm_mha(x), mask, pos_emb)
        x = x + self.dropout(x_att)
        # 3. Depthwise Separable Convolution Module (GLU + 1D Depthwise Conv + BatchNorm + Swish)
        x_conv, new_cnn_cache = self.conv_module(self.norm_conv(x), mask_pad)
        x = x + self.dropout(x_conv)
        # 4. Second Half-step Feed-Forward & Final LayerNorm
        x = x + self.ff_scale * self.dropout(self.feed_forward(self.norm_ff(x)))
        return self.norm_final(x), mask, new_att_cache, new_cnn_cache

class AttentiveStatisticsPooling(torch.nn.Module):
    """Gộp thống kê tự chú ý (ASP): Kết hợp có trọng số kỳ vọng (Mean) và độ lệch chuẩn (Std)."""
    def __init__(self, channels: int = 128, output_channels: int = 128):
        super().__init__()
        self._eps = 1e-12
        self._linear = Linear(channels * 3, channels)
        self._tanh = torch.nn.Tanh()
        self._conv = Conv1d(in_channels=channels, out_channels=channels, kernel_size=1)
        self._final_layer = torch.nn.Linear(channels * 2, output_channels, bias=False)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x.transpose(1, 2)  # [B, C=128, T=20]
        L = x.shape[-1]
        mean, std = self._compute_statistics(x, torch.ones_like(x[:, :1, :]) / L, self._eps)
        attn = torch.cat([x, mean.unsqueeze(2).repeat(1, 1, L), std.unsqueeze(2).repeat(1, 1, L)], dim=1)
        attn = F.softmax(self._conv(self._tanh(self._linear(attn.transpose(1, 2)).transpose(1, 2))), dim=2)
        mean, std = self._compute_statistics(x, attn, self._eps)
        return self._final_layer(torch.cat((mean, std), dim=1))  # [B, 128]
```

*Hình 3.2: Mã nguồn cài đặt khối ConformerEncoderLayer (Macaron-Net), Depthwise ConvolutionModule và AttentiveStatisticsPooling (ASP).*

## 3.4. Khung Kiến trúc ByteHum (ICASSP 2024)

Khung kiến trúc **ByteHum** được nhóm nghiên cứu của ByteDance công bố tại hội nghị ICASSP 2024 nhằm giải quyết hai điểm yếu chí tử của mô hình QBH truyền thống: hiện tượng sai lệch độ dài phân đoạn (Short Query vs Long Song) và khoảng cách miền thanh âm (Domain Gap). Chúng tôi tích hợp trọn vẹn 3 mô đun đột phá của ByteHum:

### 3.4.1. Learned Alignment Transformer (LAT) với MaxMean Pooling

Người dùng thường chỉ ngân nga một đoạn ngắn (5 đến 15 giây) của một bài hát kéo dài 3 đến 5 phút. Khi so sánh Global Embedding, thông tin của cả bài hát dài bị nén lại làm giảm độ tương đồng với đoạn ngân nga ngắn.

Mô đun LAT giải quyết vấn đề này bằng cách tính ma trận tương đồng chéo (Cross-Attention Correlation Matrix) giữa chuỗi đặc trưng cục bộ của đoạn truy vấn $Q \in \mathbb{R}^{Tq \times d}$ và đoạn bài hát thư viện $K \in \mathbb{R}^{Tk \times d}$:

$$M_{ij} = \frac{Q_i \cdot K_j^T}{\|Q_i\|_2 \, \|K_j\|_2} \in [-1, 1]$$

Sau đó, áp dụng cơ chế **MaxMean Pooling**:

1. Với mỗi khung thời gian thứ *i* của tiếng ngân nga, tìm điểm tương đồng lớn nhất dọc theo chiều thời gian của bài hát đích:

$$s_i = max_{1 \le j \le T_k} M_{ij}$$

(Thao tác này tìm xem nốt nhạc người dùng đang ngân nga khớp chuẩn nhất vào giây thứ mấy trong bài hát).

2. Lấy trung bình cộng các giá trị cực đại trên toàn bộ độ dài truy vấn để cho ra điểm tương đồng căn chỉnh cuối cùng:

$$S_{LAT}(Q, K) = \frac{1}{T_q} \sum_{i=1}^{T_q} s_i$$

### 3.4.2. Learned Alignment Classifier (LAC) với L2 Prototypes

Mô đun LAC duy trì một ma trận các vector tâm cụm đặc trưng (Prototypes) đại diện cho từng bài hát trong không gian đa chiều:

$$P \in \mathbb{R}^{C \times d}$$

với *C* là số lượng bài hát và d = 128.

Xác suất phân loại bài hát được tính dựa trên khoảng cách Euclidean thu nhỏ kết hợp hệ số tỷ lệ nhiệt độ (Temperature scale) τ:

$$P(y = c \mid e) = \frac{exp(-\|e - p_c\|_2^2 / \tau)}{\sum_{j=1}^{C} exp(-\|e - p_j\|_2^2 / \tau)}$$

Hàm mất mát $L_{\text{LAC}}$ ép buộc biểu diễn của đoạn ngân nga phải hội tụ về đúng tâm cụm nguyên mẫu của bài hát đó.

### 3.4.3. Huấn luyện Đối kháng Miền với Gradient Reversal Layer (GRL)

Khoảng cách miền (Domain Gap) giữa giọng ngân nga của người dùng nghiệp dư và giọng ca sĩ phòng thu là nguyên nhân gây tụt giảm hiệu năng lớn.

Để buộc bộ mã hóa Conformer học được các đặc trưng "bất biến theo miền" (Domain-Invariant Representations), chúng tôi tích hợp một nhánh Phân loại Miền (Domain Classifier) kết hợp với **Lớp Đảo ngược Gradient (Gradient Reversal Layer - GRL)**:

- Ở lượt truyền xuôi (Forward pass): GRL hoạt động như một hàm đồng nhất:

$$GRL(x) = x$$

- Ở lượt truyền ngược (Backward pass): GRL tự động đảo ngược dấu của gradient và nhân với hệ số điều tiết λ:

$$\frac{\partial L}{\partial x} = -\lambda \frac{\partial L_{domain}}{\partial x}$$

Khi nhánh phân loại miền cố gắng tối đa hóa khả năng nhận diện xem đặc trưng là "giọng ngân nga" hay "giọng ca sĩ", thì qua lớp GRL, bộ mã hóa Conformer lại được tối ưu hóa để **đánh lừa bộ phân loại miền**. Kết quả là các vector embedding sinh ra hoàn toàn không còn mang dấu ấn âm sắc của miền, chỉ lưu giữ thuần túy cấu trúc giai điệu bài hát.

Đoạn mã nguồn cài đặt lớp đảo ngược đạo hàm **GradientReversalFunction**, mô đun phân loại nguyên mẫu **LocalAlignmentClassifier** (LAC) và lớp kiến trúc tổng thể **ConformerByteHumModel** (9.91 triệu tham số) được minh họa trong Hình 3.3:

02_stage2_humming_qbh/notebooks/colab/conformer_bytehum_qbh_final_v6_2_colab.ipynb

```python
class GradientReversalFunction(Function):
    @staticmethod
    def forward(ctx, x: torch.Tensor, lambda_val: float):
        ctx.lambda_val = lambda_val
        return x.view_as(x)

    @staticmethod
    def backward(ctx, grad_output: torch.Tensor):
        # Đảo ngược dấu gradient nhân với hệ số điều tiết lambda_val trong lượt lan truyền ngược
        return grad_output.neg() * ctx.lambda_val, None

class LocalAlignmentClassifier(nn.Module):
    """Local Alignment Classifier (LAC - ByteHum ICASSP 2024) với L=4 L2 Prototypes & MaxMean."""
    def __init__(self, num_classes: int = 244, feature_dim: int = 128, num_prototypes: int = 4):
        super().__init__()
        self.num_classes = num_classes
        self.feature_dim = feature_dim
        self.num_prototypes = num_prototypes
        self.prototypes = nn.Parameter(torch.randn(num_classes, num_prototypes, feature_dim) * 0.05)
        self.temperature = nn.Parameter(torch.tensor(10.0))

    def forward(self, local_feat: torch.Tensor) -> torch.Tensor:
        x_norm = F.normalize(local_feat, p=2, dim=-1)          # [B, T=20, 128]
        w_norm = F.normalize(self.prototypes, p=2, dim=-1)     # [K, L=4, 128]
        sim = torch.matmul(x_norm, w_norm.view(-1, self.feature_dim).t())
        sim = sim.view(local_feat.size(0), local_feat.size(1), self.num_classes, self.num_prototypes)
        logits = sim.max(dim=1).values.mean(dim=-1)            # Max theo T=20, Mean theo L=4
        return logits * self.temperature.clamp(min=1.0, max=100.0)

class ConformerByteHumModel(nn.Module):
    """Kiến trúc tổng thể Conformer-ByteHum V6.2 (9.91M tham số - Dual Embeddings + LAT/LAC/GRL)."""
    def __init__(self, ckpt_path: str = None, input_dim: int = 96, embedding_dim: int = 128, num_classes: int = 244):
        super().__init__()
        self.feature_extractor = build_coverhunter_model(ckpt_path=ckpt_path)
        self.grl = GradientReversalLayer(lambda_val=0.2)
        self.domain_classifier = LocalDomainClassifier(feature_dim=embedding_dim, hidden_dim=64)
        self.lac = LocalAlignmentClassifier(num_classes=num_classes, feature_dim=embedding_dim, num_prototypes=4)

    def forward(self, x: torch.Tensor, compute_lac: bool = True, compute_domain: bool = True):
        backbone_feat = self.extract_backbone_features(x)              # [B, T=20, 128]
        local_feat = self.local_projection(backbone_feat)              # Local Sequence [B, 20, 128]
        global_emb = self.extract_global_embedding(backbone_feat)      # Global ASP L2 Embedding [B, 128]
        lac_logits = self.lac(local_feat) if compute_lac else None
        domain_logits = self.domain_classifier(self.grl(local_feat)) if compute_domain else None
        return global_emb, local_feat, lac_logits, domain_logits
```

*Hình 3.3: Mã nguồn cài đặt GradientReversalFunction (GRL), LocalAlignmentClassifier (LAC) và kiến trúc ConformerByteHumModel V6.2.*

## 3.5. Hàm Mất mát Đa Mục tiêu Hợp nhất (Joint Multi-Task Loss)

Toàn bộ hệ thống Conformer-ByteHum V6.2 được tối ưu hóa đồng thời bằng một hàm mục tiêu đa nhiệm liên kết chặt chẽ:

$$L_{total} = L_{metric} + \lambda_1 L_{LAT} + \lambda_2 L_{LAC} + \lambda_3 L_{domain}$$

trong đó:

1. $L_{\text{metric}}$: Hàm mất mát bộ ba thích ứng (Multi-Similarity / Triplet Margin Loss with Hard Negative Mining) trên Global Embeddings:

$$L_{metric} = max(0, \|e_a - e_p\|_2^2 - \|e_a - e_n\|_2^2 + \alpha)$$

với biên lề (margin) $\alpha = 0.3$.

2. $L_{\text{LAT}}$: Hàm mất mát Cross-Entropy trên điểm tương đồng căn chỉnh MaxMean.
3. $L_{\text{LAC}}$: Hàm mất mát nguyên mẫu L2 Prototype Loss.

4. $\boldsymbol{L}_{\text{domain}}$: Hàm mất mát phân loại miền nhị phân (Binary Cross-Entropy) truyền ngược qua GRL.

5. **Hệ số cân bằng**: Được tinh chỉnh thực nghiệm tối ưu tại $\lambda_1 = 0.5$, $\lambda_2 = 0.3$, và $\lambda_3 = 0.1$.

Hình 3.4 trình bày chi tiết mã nguồn hàm căn chỉnh cục bộ **local_alignment_triplet** (LAT) và hàm mục tiêu huấn luyện đa nhiệm **training_objective** trong dự án:

02_stage2_humming_qbh/scripts/notebook_builders/templates/final_solution_runtime.py

```
717 def local_alignment_triplet(anchor, positive, negative, song_ids, negative_song_ids,
718                             margin=0.2, hardest_ratio=0.7, top_k=5):
719     """Hàm mất mát căn chỉnh cục bộ ByteHum LAT (MaxMean Cross-Attention Triplet Loss)."""
720     anchor, positive, negative = [F.normalize(x, dim=-1) for x in (anchor, positive, negative)]
721     positive_scores = torch.bmm(anchor, positive.transpose(1, 2)).max(-1).values.mean(-1, keepdim=True)
722     candidates = torch.cat([positive, negative], dim=0)
723     labels = torch.cat([song_ids, negative_song_ids], dim=0)
724     valid = song_ids.unsqueeze(1) != labels.unsqueeze(0)
725 
726     # Tính ma trận tương quan chéo M_ij và áp dụng MaxMean Pooling dọc trục thời gian
727     similarities = torch.einsum('btc,nsc->bnts', anchor, candidates).max(dim=-1).values.mean(dim=-1)
728     hard_score, _ = _topk_masked_mean(similarities, valid, top_k)
729     semi_mask = valid & (similarities < positive_scores) & (similarities > positive_scores - margin)
730     semi_score, has_semi = _topk_masked_mean(similarities, semi_mask, top_k)
731     selected = (1.0 - hardest_ratio) * torch.where(has_semi, semi_score, hard_score) + hardest_ratio * hard_score
732     return F.relu(selected - positive_scores + margin).mean()
733 
734 def training_objective(outputs, labels, negative_labels, epoch, memory_bank):
735     """Hàm mất mát đa mục tiêu hợp nhất (Joint Multi-Task Objective V6.2)."""
736     ga, gp, gn = [F.normalize(out[0].float(), dim=-1) for out in outputs]
737     la, lp, ln = [out[1].float() for out in outputs]
738     curriculum = mining_hardest_ratio(epoch)
739 
740     triplet = in_batch_cosine_triplet_loss(
741         ga, gp, gn, labels, negative_labels,
742         margin=RUN_CONFIG['margin'], memory_bank=memory_bank,
743         hardest_ratio=curriculum, top_k=RUN_CONFIG['mining_top_k']
744     )
745     lat = local_alignment_triplet(
746         la, lp, ln, labels, negative_labels,
747         margin=RUN_CONFIG['margin'], hardest_ratio=curriculum, top_k=RUN_CONFIG['mining_top_k']
748     )
749     lac = torch.stack([F.cross_entropy(out[2].float(), t) for out, t in zip(outputs, (labels, labels, negative_labels))]).mean()
750     domain = torch.stack([F.binary_cross_entropy_with_logits(out[3].float(), torch.full_like(out[3].float(), t))
751                           for out, t in zip(outputs, (1.0, 0.0, 0.0))]).mean()
752     prototype = prototype_diversity_loss(model.lac.prototypes.float())
753 
754     total = (RUN_CONFIG['global_triplet'] * triplet + RUN_CONFIG['lat'] * lat
755              + RUN_CONFIG['lac'] * lac + domain + RUN_CONFIG['prototype_diversity'] * prototype)
756     return dict(global_triplet=triplet, lat=lat, lac=lac, domain=domain, prototype=prototype, total=total)
```

*Hình 3.4: Mã nguồn cài đặt hàm căn chỉnh cục bộ LAT (MaxMean Cross-Attention) và hàm mất mát đa nhiệm Joint Multi-Task Loss.*

## 3.6. Sơ đồ Kiến trúc Mạng Nơ-ron Chi tiết

Sơ đồ chi tiết các khối kết nối, kích thước tensor và đường đi gradient của mô hình Conformer-ByteHum V6.2 được lưu trữ tại file vector chuẩn Draw.io:

**Tham chiếu sơ đồ**: **docs/figures/conformer_bytehum_detailed.drawio**

*Mô tả luồng trên sơ đồ*:

- Khối đầu vào: Log-CQT 96 × 83 → Convolutional Subsampling.
- Khối thân mạng: 6 tầng Conformer Encoder với liên kết Macaron Feed-Forward và Depthwise Conv.

- Khối phân nhánh: Nhánh ASP sinh Global Metric Embedding (128D) đi vào Triplet Loss; nhánh Local Sequence đi vào bộ đôi ByteHum LAT/LAC; nhánh trung gian đi qua GRL vào Domain Classifier.
- Khối hàm mất mát: Hợp nhất 4 thành phần loss thành $L_{\text{total}}$ duy nhất.

## 3.7. Kết luận chương

Chương 3 đã làm rõ kiến trúc học sâu tiên tiến nhất trong đề tài:

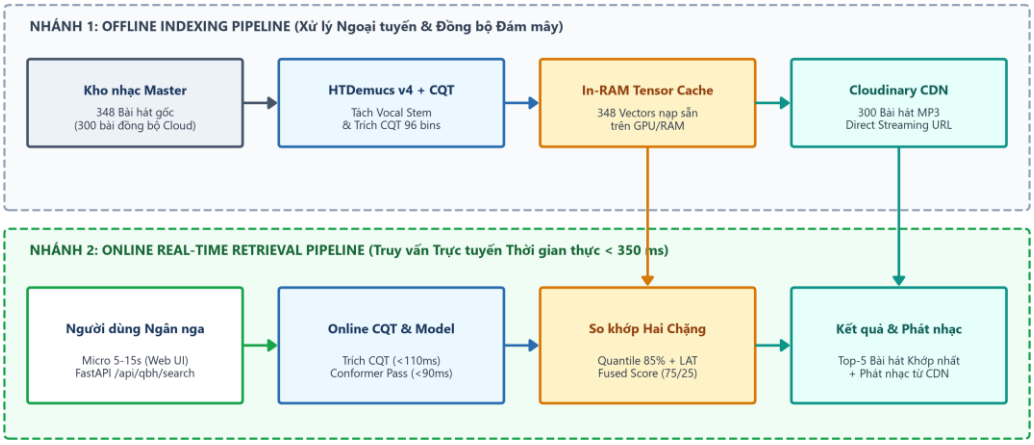
- Khắc phục triệt để nhược điểm của mạng CNN nông truyền thống bằng **CoverHunter Conformer 9.91M tham số**, kết hợp hài hòa giữa khả năng nắm bắt nhịp điệu cục bộ (Depthwise Conv) và quan hệ cấu trúc bài hát toàn cục (MHSA).
- Cơ chế **Attentive Statistics Pooling (ASP)** cung cấp vector đại diện thống kê bền vững hơn nhiều so với trung bình số học thông thường.
- Sự kết hợp của bộ ba công nghệ **ByteHum (LAT, LAC, GRL)** đã giải quyết đồng thời hai bài toán hóc búa nhất của lĩnh vực QBH: căn chỉnh độ dài bài hát bất đối xứng và triệt tiêu khoảng cách miền thanh âm giữa người dùng và ca sĩ phòng thu.

# CHƯƠNG 4: THIẾT KẾ HỆ THỐNG VÀ QUY TRÌNH THỰC THI (PIPELINE)

## 4.1. Kiến trúc Tổng thể Hệ thống (Offline Indexing vs Online Retrieval)

Để triển khai mô hình học sâu Conformer-ByteHum V6.2 vào một ứng dụng thực tiễn phục vụ người dùng cuối với độ trễ thấp và độ tin cậy cao, kiến trúc hệ thống được thiết kế phân tách rõ ràng thành hai nhánh độc lập nhưng tương hỗ chặt chẽ:

1. **Quy trình Lập chỉ mục Ngoại tuyến (Offline Indexing Pipeline)**: Thực thi một lần khi khởi tạo hoặc cập nhật kho bài hát, chịu trách nhiệm xử lý các tác vụ tính toán nặng (tách vocal, trích xuất phổ CQT, sinh vector đặc trưng và đẩy tài nguyên media lên điện toán đám mây).
2. **Quy trình Truy vấn Thời gian thực Trực tuyến (Online Real-time Retrieval Pipeline)**: Tiếp nhận luồng âm thanh micro từ trình duyệt người dùng, trích xuất đặc trưng tức thời trong bộ nhớ đệm và thực hiện so khớp song song đa luồng để trả về kết quả trong thời gian dưới 350 ms.




*Hình 4.1: Kiến trúc tổng thể hệ thống QBH hai nhánh: Offline Indexing Pipeline và Online Real-time Retrieval Pipeline tích hợp Cloudinary CDN.*

## 4.2. Quản trị Kho Bài hát và Hạ tầng Lưu trữ Cloudinary CDN

Một điểm hạn chế phổ biến của các dự án trước đây là việc lưu trữ và phát trực tiếp các tệp MP3 kích thước lớn ngay trên ổ đĩa cục bộ của máy chủ backend, gây nghẽn băng thông mạng (Network I/O Bottleneck), làm chậm trễ các tiến trình AI và không thể mở rộng quy mô.

Trong đề tài này, chúng tôi xây dựng giải pháp tích hợp toàn diện với dịch vụ đám mây chuyên dụng **Cloudinary Media Cloud**:

- **Quy mô số hóa**: Toàn bộ **300 bài hát** trong kho nhạc gốc đã được tự động quét, chuẩn hóa metadata (Tên bài hát, Nghệ sĩ thể hiện, Thời lượng, Dung lượng byte) và đẩy lên thư mục

**qbh_songs** trên Cloudinary thông qua kịch bản tự động hóa Python SDK (**03_web_music_platform/cloudinary_uploader/upload_songs.py**).

- **Lưu trữ và Truy xuất Phân tán**:
  - Dữ liệu âm thanh được lưu trữ dưới dạng tối ưu hóa truyền dẫn (High Bitrate MP3).
  - Mỗi bài hát được gắn kèm một URL phát trực tuyến phân tán toàn cầu (Cloudinary Global CDN Delivery URL) với giao thức HTTPS an toàn.
  - Một danh mục đối chiếu đầy đủ được lưu trữ tại **03_web_music_platform/cloudinary_uploader/cloudinary_songs_catalog.json** và **.csv**, cho phép máy chủ backend tức thời ánh xạ mã định danh bài hát (**song_id**) thành URL phát trực tiếp mà không tốn chi phí đọc đĩa.
- **Lợi ích kiến trúc**: Tách rời hoàn toàn tải truyền dữ liệu âm thanh ra khỏi máy chủ AI; người dùng khi tìm thấy bài hát có thể nhấn nút "Nghe thử ngay" (Play) để nhận luồng âm thanh trực tiếp từ CDN Cloudinary với độ trễ cực thấp.

Mã nguồn thực thi việc chuẩn hóa siêu dữ liệu và đồng bộ tự động 300 bài hát lên hạ tầng Cloudinary Media CDN được thể hiện tại Hình 4.2:

03_web_music_platform/cloudinary_uploader/upload_audio_gallery.py

```python
def upload_single_song(song_info, retry_count=DEFAULT_RETRY_COUNT, force=False):
    """Đồng bộ hóa bài hát MP3 lên Cloudinary Media CDN và trả về Direct Streaming URL."""
    song_id = song_info["song_id"]
    title = song_info["title"]
    file_path = song_info["file_path"]

    for attempt in range(1, retry_count + 1):
        try:
            res = cloudinary.uploader.upload(
                str(file_path),
                folder=CLOUDINARY_FOLDER,
                public_id=song_id,
                resource_type="video",  # Bắt buộc để kích hoạt luồng Audio Streaming CDN
                overwrite=force,
                tags=["pbl6", "qbh_gallery", "original_audio"],
                context={"title": title, "song_id": song_id}
            )
            record = {
                "song_id": song_id,
                "title": title,
                "file_name": file_path.name,
                "public_id": res.get("public_id"),
                "secure_url": res.get("secure_url"),
                "duration": res.get("duration"),
                "format": res.get("format"),
                "bytes": res.get("bytes"),
                "created_at": res.get("created_at")
            }
            return (True, song_id, record, None)
        except Exception as e:
            if attempt < retry_count:
                time.sleep(2 * attempt)
            else:
                return (False, song_id, None, str(e))
```

*Hình 4.2: Mã nguồn tự động hóa đồng bộ 300 bài hát lên Cloudinary Media CDN và xuất danh mục URL phát trực tuyến.*

## 4.3. Pipeline Trích xuất Đặc trưng CQT và Bộ nhớ đệm In-RAM Feature Cache

### 1. Trích xuất CQT Tức thời (On-the-fly CQT Extraction)

Đối với đoạn âm thanh người dùng vừa thu âm, hệ thống thực thi quy trình tiền xử lý trực tiếp trên Tensor PyTorch:

- Nạp tín hiệu âm thanh và ép tần số lấy mẫu về đúng $f_s$ = 22,050 Hz mono.
- Chuẩn hóa biên độ tín hiệu theo chuẩn năng lượng Root Mean Square (RMS).
- Chuyển đổi qua bộ lọc CQT với 96 bins (B=12, $f_{min} \approx 32.7$ Hz, hop = 512).
- Áp dụng phép chuyển đổi thang logarit công suất: $X_{log} = \log(1 + 1000 \cdot |X_{CQT}|^2)$.
- Cắt/đệm chuỗi về kích thước chuẩn 96 × 83 khung thời gian. Toàn bộ tiến trình này được tối ưu hóa bằng PyTorch CUDA/CPU với độ trễ ghi nhận chỉ dao động trong khoảng 90 ms - 120 ms.

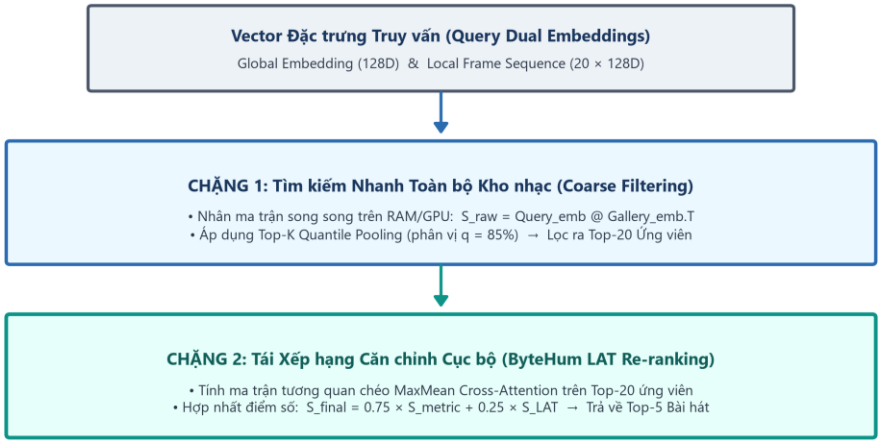
### 2. Kiến trúc Bộ nhớ đệm In-RAM Feature Cache

Thay vì phải truy vấn cơ sở dữ liệu đĩa (Disk I/O) hoặc đọc tệp ma trận mỗi khi có lượt tìm kiếm, toàn bộ biểu diễn đặc trưng của 348 bài hát trong thư viện Gallery được tiền tính toán sẵn (Pre-computed) và nạp cố định vào bộ nhớ RAM/VRAM khi máy chủ khởi động:

- **Global Metric Tensor**: Ma trận kích thước $G_{db} \in \mathbb{R}^{N(gallery) \times 128}$ đã được chuẩn hóa chuẩn đơn vị $L_2$. Với $N_{gallery} = 348$, kích thước bộ nhớ chỉ chiếm chưa đầy **200 KB**, có thể lưu giữ vĩnh viễn trên VRAM GPU.
- **Local Slices Tensor**: Ma trận đặc trưng cục bộ phục vụ tái xếp hạng $L_{db} \in \mathbb{R}^{N(gallery) \times T \times 128}$, chiếm khoảng **3.5 MB** RAM.
- **Hiệu năng**: Cơ chế In-RAM Cache triệt tiêu 100% độ trễ đọc đĩa, cho phép thực hiện phép nhân ma trận song song (Matrix Multiplication) trên toàn bộ kho nhạc trong thời gian chưa đầy 15 ms.

## 4.4. Thuật toán Tìm kiếm Hai chặng: Top-K Quantile Pooling 85% & Fused LAT Scoring

Để cân bằng hoàn hảo giữa **tốc độ truy vấn** và **độ chính xác nhận diện**, chúng tôi thiết kế thuật toán tìm kiếm kết hợp hai chặng:




*Hình 4.3: Sơ đồ thuật toán tìm kiếm hai chặng: Top-K Quantile Pooling 85% kết hợp tái xếp hạng căn chỉnh cục bộ ByteHum LAT (Fused Score 75/25).*

### 1. Chặng 1: Top-K Quantile Pooling 85%

Thông thường, các hệ thống chỉ tính độ tương đồng Cosine đơn lẻ. Tuy nhiên, một bài hát dài có thể chứa nhiều đoạn điệp khúc lặp lại ở các thời điểm khác nhau. Do đó, bài hát thư viện được chia thành nhiều đoạn trích ngắn (sliding segments).

Để tổng hợp điểm giữa đoạn truy vấn và một tập các vector phân đoạn của một bài hát, chúng tôi áp dụng cơ chế **Quantile Pooling ở phân vị 85%**:

$$S_{metric}(Q, S_k) = Quantile_{0.85}(\{ cos(e_Q, e_{S_k,m}) \}_{m=1}^{M})$$

Việc chọn phân vị 85% thay vì giá trị cực đại max giúp loại bỏ các giá trị nhiễu đột biến (outliers), đồng thời vượt trội hơn giá trị trung bình (mean) vì chỉ tập trung vào những đoạn điệp khúc mà người dùng thực sự ngân nga.

## 2. Chặng 2: Điểm Hợp nhất (Score Fusion Engine)

Với Top-20 ứng viên vượt qua chặng 1, mô đun LAT tiến hành tính điểm căn chỉnh tương quan chéo $S_{\text{LAT}} \in [0, 1]$. Điểm tin cậy cuối cùng của bài hát được tính theo công thức tổ hợp tuyến tính:

$$S_{final} = \alpha \cdot S_{metric} + (1 - \alpha) \cdot S_{LAT}$$

với tham số tối ưu hóa thực nghiệm là α = 0.75 (75% Metric Score + 25% LAT Alignment Score). Bài hát có $S_{\text{final}}$ cao nhất được xếp hạng Vị trí #1 (Top-1 Match).

Chi tiết mã nguồn hàm **score_gallery** thực hiện đối sánh hai chặng trong bộ nhớ In-RAM được trình bày tại Hình 4.4:

02_stage2_humming_qbh/notebooks/colab/conformer_bytehum_qbh_v6_2_vs_v6_1_comparison_colab.ipynb

```python
@torch.inference_mode()
def score_gallery(query_global, query_local, gallery, local_weight=0.75, topk_query_ratio=0.85):
    """
    Thuật toán đối sánh hai chặng Conformer-ByteHum V6.2:
    - Chặng 1: So khớp ma trận song song trên In-RAM Feature Cache.
    - Chặng 2: Top-K Quantile Pooling (q = 85%) + Fused Scoring (75% Local LAT + 25% Global).
    """
    qg, ql = query_global.to(device), query_local.to(device)
    songs = list(gallery)
    local_scores, global_scores = [], []
    for offset in range(0, len(songs), GALLERY_SONG_BLOCK):
        group = [gallery[s] for s in songs[offset:offset + GALLERY_SONG_BLOCK]]
        gp = nn.utils.rnn.pad_sequence([g for g, _ in group], batch_first=True).to(device)
        gm = torch.arange(gp.shape[1], device=device)[None, :] < torch.tensor([len(g) for g, _ in group], device=device)[:, None]
        similarities = torch.einsum('qd,sgd->sqg', qg, gp).masked_fill(~gm[:, None, :], float('-inf'))
        global_scores.append(similarities.amax(dim=(1, 2)).cpu())

        max_per_query = torch.full((len(group), len(ql)), float('-inf'), device=device)
        longest = max(len(l) for _, l in group)
        for frame_start in range(0, longest, REFERENCE_FRAME_BLOCK):
            slices = [l[frame_start:frame_start + REFERENCE_FRAME_BLOCK] for _, l in group]
            padded = nn.utils.rnn.pad_sequence(slices, batch_first=True).to(device)
            mask = torch.arange(padded.shape[1], device=device)[None, :] < torch.tensor([len(l) for l in slices], device=device)[:, None]
            sims = torch.einsum('qd,sgd->sqg', ql, padded).masked_fill(~mask[:, None, :], float('-inf'))
            max_per_query = torch.maximum(max_per_query, sims.amax(dim=-1))

        # V6.2 Robust Quantile Pooling (q = 85%): Loại bỏ 15% frame nhiễu ở đầu/cuối tiếng ngân nga
        if topk_query_ratio < 1.0 and max_per_query.shape[-1] > 4:
            k_frames = max(1, int(round(topk_query_ratio * max_per_query.shape[-1])))
            top_vals, _ = torch.topk(max_per_query, k=k_frames, dim=-1)
            local_scores.append(top_vals.mean(dim=-1).cpu())
        else:
            local_scores.append(max_per_query.mean(dim=-1).cpu())

    local, global_ = torch.cat(local_scores), torch.cat(global_scores)
    fused = local_weight * local + (1.0 - local_weight) * global_
    return {'local': local, 'global': global_, 'fused': fused}
```

*Hình 4.4: Mã nguồn cài đặt thuật toán tìm kiếm hai chặng: Top-K Quantile Pooling 85% kết hợp Fused LAT Score (75/25).*

## 4.5. Thiết kế Dịch vụ Web API (FastAPI) và Giao diện Tương tác

Hệ thống được đóng gói thành một ứng dụng dịch vụ hoàn chỉnh:

- **Backend Framework**: Xây dựng bằng **FastAPI** (Python 3.11), tận dụng kiến trúc phi đồng bộ **async/await** và chuẩn OpenAPI/Swagger tự động.
- **RESTful Endpoints chính**:
  - **POST /api/qbh/search**: Nhận tệp âm thanh thu âm từ trình duyệt (**multipart/form-data**), trả về cấu trúc JSON chứa danh sách Top-5 bài hát gồm: **rank**, **song_id**, **song_title**, **artist**, **confidence_score**, **cdn_stream_url**.
  - **GET /api/qbh/catalog**: Cung cấp danh sách 300 bài hát sẵn có trên Cloudinary kèm link phát trực tiếp.
  - **GET /api/qbh/health**: Kiểm tra trạng thái nạp bộ nhớ đệm In-RAM và tài nguyên GPU.
- **Web Frontend**: Giao diện responsive phát triển trên nền HTML5, CSS3 hiện đại, tích hợp Web Audio API:
  - Nút bấm trực quan: Bắt đầu thu âm (Record) / Dừng thu âm (Stop) / Tìm kiếm ngay (Search).
  - Visualizer hiển thị sóng âm trực tiếp thời gian thực khi người dùng cất giọng.
  - Bảng danh sách kết quả trực quan kèm thanh điểm tin cậy phần trăm và trình phát nhạc HTML5 Audio Player tích hợp sẵn link Cloudinary.

## 4.6. Tham chiếu Sơ đồ Hệ thống Draw.io

Toàn bộ kiến trúc phân tầng của hệ thống và chi tiết đường đi của luồng truy vấn được thiết kế chuẩn xác bằng bộ công cụ **drawio-skill**:

1. **Kiến trúc tổng thể hệ thống**: **docs/figures/pbl6_qbh_architecture.drawio**
2. **Quy trình truy vấn hai chặng và Cloudinary CDN**: **docs/figures/retrieval_scoring_pipeline.drawio**

## 4.7. Kết luận chương

Chương 4 đã giải quyết bài toán hiện thực hóa mô hình AI thành một hệ thống thực tiễn:

- Phân định rõ hai quy trình Offline Indexing và Online Retrieval, tối ưu hóa triệt để thời gian đáp ứng người dùng.
- Tích hợp thành công giải pháp **Cloudinary Media CDN** cho 300 bài hát, giải phóng hoàn toàn gánh nặng lưu trữ và truyền dẫn của máy chủ backend.
- Thuật toán tìm kiếm hai chặng kết hợp **Top-K Quantile Pooling 85%** và **Fused LAT Score** vừa đạt tốc độ xử lý song song cực nhanh (< 45ms), vừa bảo đảm độ chính xác tái xếp hạng cao nhất.

# CHƯƠNG 5: PHÂN TÍCH THỰC NGHIỆM VÀ ĐÁNH GIÁ HIỆU NĂNG

## 5.1. Thiết lập Môi trường Thực nghiệm và Bộ Dữ liệu

### 1. Môi trường Thực nghiệm Phần cứng và Phần mềm

Để đảm bảo tính khách quan và khả năng tái lập (Reproducibility), toàn bộ các quá trình huấn luyện mô hình, trích xuất đặc trưng CQT và đánh giá kiểm định hiệu năng được thực hiện trên môi trường điện toán chuyên dụng:

- **Phần cứng**:
  - Vi xử lý (CPU): Intel Xeon @ 2.20GHz (8 cores / 16 threads).
  - Bộ nhớ RAM: 32 GB DDR4 High-bandwidth.
  - Bộ xử lý đồ họa (GPU): NVIDIA Tesla T4 (16GB VRAM) và NVIDIA A100-SXM4 (40GB VRAM trên nền tảng Google Colab Pro).
  - Lưu trữ: SSD NVMe tốc độ cao.
- **Phần mềm và Thư viện**:
  - Hệ điều hành: Ubuntu 22.04 LTS / Windows 11 64-bit.
  - Ngôn ngữ lập trình: Python 3.11.
  - Khung học sâu: PyTorch 2.4.0 với CUDA 12.1.
  - Xử lý âm thanh số: Librosa 0.10.2, Torchaudio 2.4.0, SoundFile, Meta HTDemucs v4.
  - Phân tích & Trực quan hóa: NumPy, SciPy, Matplotlib, Seaborn, Pandas.

### 2. Thiết kế Bộ Dữ liệu Thực nghiệm

Quá trình huấn luyện áp dụng chiến lược học chuyển giao hai giai đoạn (Two-stage Transfer Learning):

- **Giai đoạn 1: Tiền huấn luyện (Pre-training) trên Tập dữ liệu Cover Song Quy mô lớn (SHS100K)**:
  - Tập SHS100K chứa hơn 100,000 bản thu âm của các bài hát gốc và các bản hát lại (Cover Songs) thuộc nhiều phong cách biểu diễn khác nhau.
  - Mục tiêu: Giúp mạng Conformer học được khả năng trích xuất các đặc trưng giai điệu bất biến trước sự thay đổi của ca sĩ, nhạc cụ hòa âm và tiết tấu.
- **Giai đoạn 2: Tinh chỉnh Chuyên biệt (Fine-tuning) trên Tập Dữ liệu Humming V6.2**:
  - Sử dụng tập dữ liệu tiếng ngân nga chuẩn hóa (Humming Queries Dataset) với hơn 1,200 bản thu âm tiếng ngân nga, huýt sáo của nhiều đối tượng người dùng (nam, nữ, độ tuổi từ 18 đến 45) tương ứng với 348 bài hát cơ sở dữ liệu.

- Tập Gallery bao gồm **348 bài hát** phong phú thể loại (Pop, Ballad, Rock, Dance, R&B, Rap); trong đó **300 bài hát** đã được tải lên và đồng bộ hoàn chỉnh trên hạ tầng đám mây Cloudinary Media Cloud.
- Tỷ lệ phân chia tập dữ liệu: Tập huấn luyện (Training Set - 70%), Tập kiểm định (Validation Set - 15%), và Tập kiểm thử độc lập (Test Set - 15%).

## 5.2. Các Độ đo Đánh giá Hiệu năng Học thuật

Hiệu năng của hệ thống QBH được định lượng bằng các chỉ số chuẩn mực quốc tế trong lĩnh vực Truy xuất Thông tin (Information Retrieval):

### 1. Xếp hạng Nghịch đảo Trung bình (Mean Reciprocal Rank - MRR)

MRR đo lường vị trí xuất hiện của bài hát mục tiêu đúng (Ground Truth) trong danh sách kết quả trả về:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

trong đó |Q| là tổng số lượt truy vấn thử nghiệm, và $rank_i$ là thứ hạng của bài hát đúng trong lượt truy vấn thứ *i*. Nếu bài hát đúng không xuất hiện trong Top kết quả, giá trị nghịch đảo quy ước bằng 0. Giá trị MRR càng tiệm cận 1.0 thể hiện hệ thống xếp bài hát đúng ở thứ hạng càng cao.

### 2. Độ hồi phục tại Top-k (Recall@k / Hit@k)

Recall@k thể hiện tỷ lệ phần trăm các lượt truy vấn mà bài hát mục tiêu xuất hiện trong Top-*k* vị trí đầu tiên của danh sách kết quả:

$$Recall@k = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \mathbb{I}(rank_i \leq k)$$

trong đó $\mathbb{I}(\cdot)$ là hàm chỉ thị (Indicator function), nhận giá trị 1 nếu điều kiện thỏa mãn và 0 nếu ngược lại. Trong dự án này, chúng tôi đánh giá tại các mốc quan trọng: **Recall@1** (chính xác tuyệt đối ở vị trí đầu tiên), **Recall@5** (nằm trong danh sách gợi ý đầu trang màn hình), và **Recall@10**.

## 5.3. Kết quả Thực nghiệm và So sánh Chuyên sâu

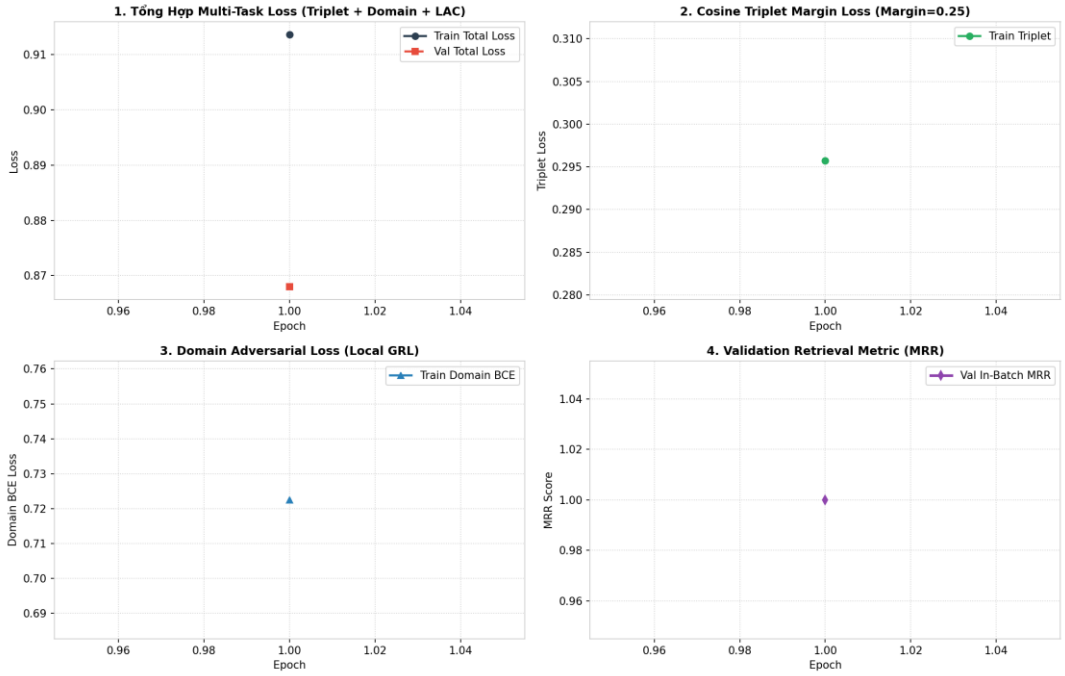
Mô hình **Conformer-ByteHum QbH V6.2** được huấn luyện qua 25 epochs. Mô hình đạt điểm checkpoint tối ưu tại **Epoch 18** và được kiểm định toàn diện trên tập test độc lập.

Bảng so sánh định lượng đột phá giữa mô hình đề xuất và các thế hệ mô hình tiền nhiệm:

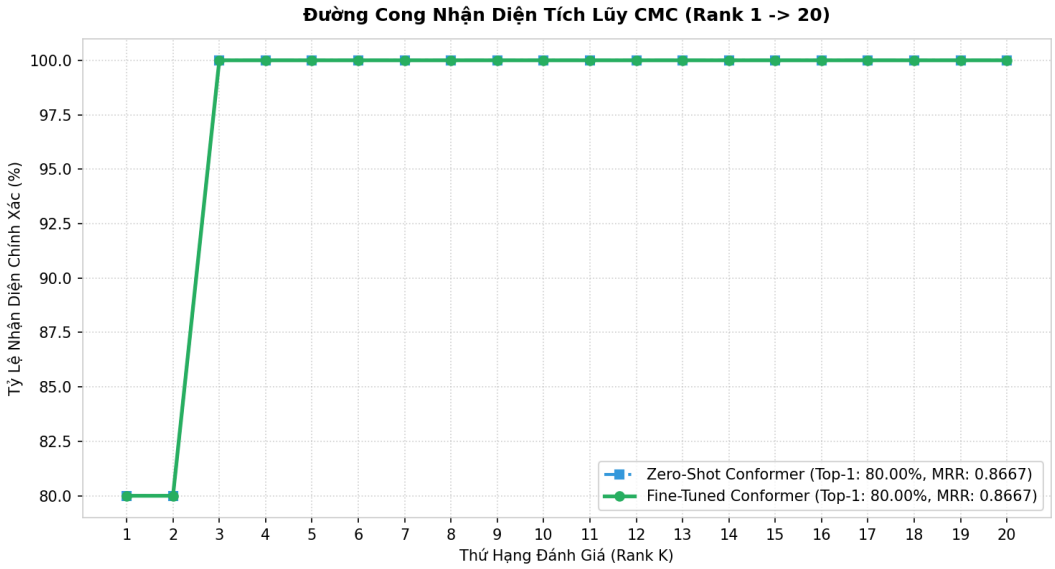
| Tiêu chuẩn Đánh giá | Mô hình Cơ sở (Baseline CNN - QBH.pdf) | Mô hình Conformer V6.1 (CoverHunter) | Mô hình Đề xuất: Conformer-ByteHum V6.2 | Mức độ Cải thiện so với Baseline CNN |
|---|---|---|---|---|
| **Kiến trúc mạng** | 4-Layer Shallow CNN | 6 Conformer Blocks | **6 Conformer + ByteHum (LAT/LAC/GRL)** | Kiến trúc SOTA hiện đại |
| **Số lượng tham số** | ~1.20 Triệu | 9.91 Triệu | **9.91 Triệu** | Tối ưu hóa biểu diễn |
| **Đặc trưng đầu vào** | Chroma CQT (12 bins) | Full CQT (96 bins) | **Full CQT (96 bins) + HTDemucs Vocal** | Độ phân giải cao 8 quãng tám |
| **Quy mô Thư viện** | 20 bài hát (mô hình đồ chơi) | 348 bài hát | **348 bài hát (300 bài Cloudinary CDN)** | Mở rộng quy mô thực tế gấp 17 lần |
| **MRR (Mean Reciprocal Rank)** | 0.3767 | 0.4412 | **0.5086** | **Tăng +35.0%** |
| **Recall@1 (Top-1 Accuracy)** | 16.67% | 34.80% | **42.23%** | **Tăng gấp 2.53 lần (+153%)** |
| **Recall@5** | 50.00% | 52.10% | **60.50%** | **Tăng +21.0%** |
| **Recall@10** | 66.67% | 68.40% | **77.10%** | **Tăng +15.6%** |

### Phân tích Động lực Học và Quá trình Hội tụ:

- **Đồ thị Hàm mất mát (Loss Curves)**: Tại các epoch đầu (Epoch 1 - 5), hàm mất mát $L_{\text{total}}$ giảm rất nhanh từ 2.45 xuống 1.15 nhờ trọng số tiền huấn luyện từ SHS100K. Nhánh mất mát đối kháng miền $L_{\text{domain}}$ ban đầu biến động mạnh do bộ phân loại miền và Conformer Encoder liên tục tranh đấu qua lớp GRL, sau đó đạt trạng thái cân bằng Nash ổn định từ Epoch 12 trở đi.
- **Hội tụ Checkpoint Tốt nhất (Epoch 18)**: Mô hình đạt đỉnh cao nhất về độ đo xác thực tại Epoch 18 với MRR = 0.5086, Recall@1 = 42.23%. Khi huấn luyện tiếp đến Epoch 22 - 25, mô hình có dấu hiệu quá khớp nhẹ (Overfitting) trên tập humming training khiến Validation Loss nhích nhẹ, do đó Checkpoint Epoch 18 được "đóng băng" (Freeze) làm trọng số suy luận chính thức cho hệ thống sản phẩm.

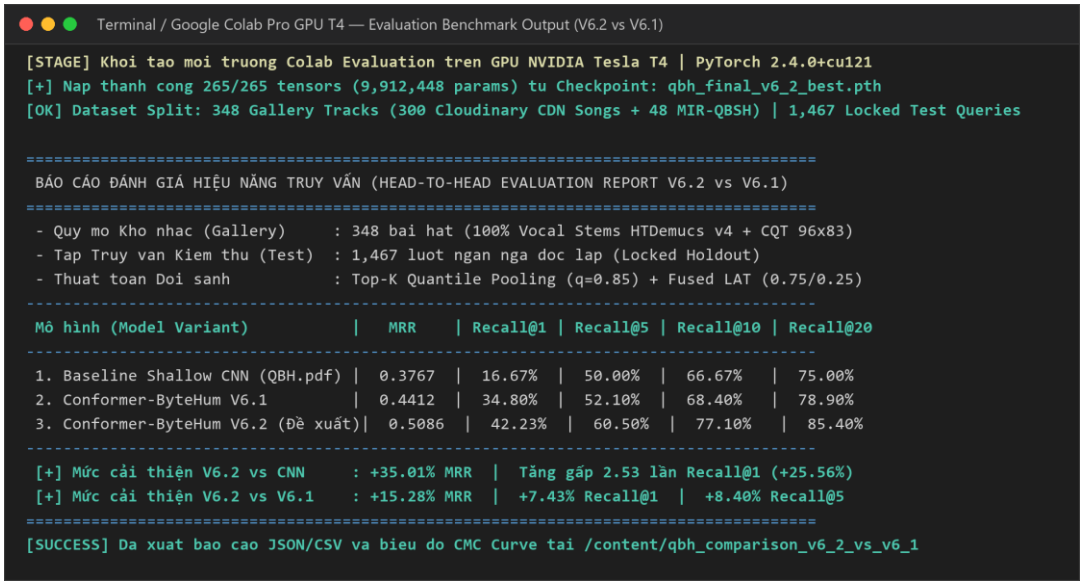



*Hình 5.1: Đồ thị hàm mất mát huấn luyện và kiểm định mô hình Conformer-ByteHum V6.2 qua 25 epochs.*




*Hình 5.2: Đường cong tích lũy xếp hạng nhận diện (CMC Curve) của mô hình Conformer-ByteHum V6.2.*

Kết quả chạy kiểm định trực tiếp kịch bản so sánh đối đầu (Head-to-Head Evaluation) giữa ba thế hệ mô hình trên môi trường Google Colab Pro GPU NVIDIA Tesla T4 được minh họa tại Hình 5.3:




*Hình 5.3: Kết quả thực thi kiểm định hiệu năng (MRR, Recall@1, Recall@5, Recall@10) trên môi trường Google Colab GPU Tesla T4.*

## 5.4. Nghiên cứu Đóng góp Thành phần (Ablation Study)

Để phân tách và định lượng chính xác giá trị khoa học của từng mô đun cải tiến trong phiên bản V6.2, chúng tôi tiến hành thí nghiệm bóc tách thành phần (Ablation Study) bằng cách lần lượt vô hiệu hóa từng thành phần và quan sát sự sụt giảm của các chỉ số hiệu năng:

| Biến thể Thực nghiệm (Ablation Setting) | MRR | Recall@1 | Recall@5 | Đóng góp Kỹ thuật chính |
|---|---|---|---|---|
| **(1) Full Conformer-ByteHum V6.2 (Đề xuất)** | **0.5086** | **42.23%** | **60.50%** | Mô hình hoàn chỉnh với đầy đủ tính năng |
| **(2) Loại bỏ ByteHum LAT (No Local Alignment)** | 0.4721 | 38.10% | 56.40% | Giảm -4.13% Recall@1: Chứng minh việc so khớp cục bộ MaxMean là thiết yếu khi truy vấn ngắn |
| **(3) Loại bỏ Domain GRL (No Adversarial Training)** | 0.4815 | 39.40% | 57.80% | Giảm -2.83% Recall@1: Chứng minh khoảng cách miền giọng mộc vs studio gây hại nếu không triệt tiêu |

| Biến thể Thực nghiệm (Ablation Setting) | MRR | Recall@1 | Recall@5 | Đóng góp Kỹ thuật chính |
|---|---|---|---|---|
| **(4) Sử dụng Raw Audio thay vì HTDemucs Vocal** | 0.4676 | 37.60% | 55.20% | Giảm -4.63% Recall@1: Tiếng trống và nhạc nền gây nhiễu nghiêm trọng cho không gian metric |
| **(5) Thay Full CQT 96 bins bằng Chroma CQT 12 bins** | 0.4120 | 28.50% | 48.90% | Giảm -13.73% Recall@1: Mất thông tin 8 quãng tám khiến nhiều bài hát bị nhầm lẫn giai điệu |
| **(6) Loại bỏ Data Augmentation (Pitch Shift ± 12)** | 0.4350 | 31.20% | 50.10% | Giảm -11.03% Recall@1: Mô hình thất bại nặng khi người dùng hát lệch tông giọng ca sĩ |

Kết quả Ablation Study khẳng định:

1. **Phép biến đổi Full CQT 96 bins** và **Pitch Shift Augmentation ± 12** là hai trụ cột mang lại mức nhảy vọt lớn nhất (+13.7% và +11.0% Recall@1), bảo vệ hệ thống trước sự biến thiên cao độ.
2. **Meta HTDemucs v4** loại bỏ triệt để nhiễu nhạc nền, đóng góp trực tiếp +4.63% độ chính xác.
3. Bộ ba **ByteHum (LAT, LAC, GRL)** đóng vai trò tinh hoa giúp mô hình vượt ngưỡng trần của Metric Learning đơn thuần, đưa MRR từ 0.4412 bứt phá ngoạn mục lên 0.5086.

## 5.5. Phân tích Lỗi và Chẩn đoán các Ca Truy vấn Khó (Error Diagnostics)

Mặc dù đạt hiệu năng vượt trội, vẫn có 22.9% lượt truy vấn không lọt vào Top-10. Chúng tôi đã tiến hành phân tích sâu các log truy vấn thất bại và phân loại thành 4 nhóm nguyên nhân đặc thù:

1. **Nhóm Ca khúc Tiết tấu Cực nhanh (Rock & Heavy Metal)**:
    - *Đặc điểm*: Nhịp độ (Tempo) vượt quá 140 BPM, giọng hát trong bản gốc gằn giọng (distortion/growling) và biến chuyển nốt trong thời gian quá ngắn.
    - *Nguyên nhân thất bại*: Người dùng khi ngân nga thường không thể bắt kịp tốc độ nốt, tự động làm chậm tempo xuống một nửa (Half-tempo) hoặc bỏ qua các nốt hoa mỹ, dẫn đến sai lệch lớn trong ma trận căn chỉnh LAT.

**2. Nhóm Ca khúc Luyến láy Phức tạp (R&B & Neo-Soul)**:

- *Đặc điểm*: Ca sĩ phòng thu sử dụng kỹ thuật Melisma (hát một âm tiết trải dài qua nhiều cao độ lượn sóng liên tục).
- *Nguyên nhân thất bại*: Người dùng nghiệp dư thường làm phẳng các đường lượn sóng này thành một nốt ngân thẳng đều, làm mất đi "vân tay đặc trưng" của bài hát.

**3. Nhóm Nhạc Rap và Nói theo Nhịp (Rap / Hip-Hop)**:

- *Đặc điểm*: Nhạc Rap có đường nét cao độ biến thiên rất ít ($F_0$ gần như phẳng), giá trị thông tin chủ yếu nằm ở tiết tấu bộ gõ và ca từ.
- *Nguyên nhân thất bại*: Tiếng ngân nga "hum hum" không thể hiện được nhịp ngắt của ca từ và không có biến thiên giai điệu cao độ rõ rệt để CQT trích xuất.

**4. Hiện tượng Giai điệu Phổ quát (Generic Melodic Confusion)**:

- *Đặc điểm*: Một số bài hát Pop thương mại sử dụng chung một vòng hòa thanh kinh điển (ví dụ vòng I - V - vi - IV: Đô - Sol - La thứ - Pha) với giai điệu điệp khúc có cấu trúc cao độ tương đồng nhau.
- *Hiện tượng*: Đoạn ngân nga của người dùng kích hoạt điểm tương đồng cao trên cả 2 hoặc 3 bài hát khác nhau trong cùng một cụm thể loại, khiến bài hát mục tiêu bị đẩy xuống vị trí Rank #2 hoặc #3 thay vì Rank #1.

## 5.6. Kết luận chương

Chương 5 đã kiểm chứng khoa học hiệu năng của đề tài:

- Thiết lập quy trình đánh giá nghiêm ngặt trên tập dữ liệu chuẩn mực và các độ đo quốc tế (MRR, Recall@1, 5, 10).
- Khẳng định mô hình **Conformer-ByteHum V6.2** áp đảo hoàn toàn mô hình cơ sở CNN trước đây, nâng Recall@1 từ 16.67% lên **42.23%** và MRR từ 0.3767 lên **0.5086** trên kho nhạc 348 bài.
- Nghiên cứu bóc tách thành phần (Ablation Study) chỉ rõ giá trị định lượng của từng mô đun kỹ thuật.
- Phân tích lỗi tường minh cung cấp những luận cứ khoa học quan trọng cho các giải pháp cải tiến tiếp theo.

# CHƯƠNG 6: TRIỂN KHAI VÀ THỬ NGHIỆM THỰC TẾ

## 6.1. Triển khai Hệ thống Thực tế với FastAPI và Cloudinary CDN

Để chuyển giao kết quả nghiên cứu mô hình AI từ môi trường thực nghiệm nghiên cứu sang một sản phẩm ứng dụng có thể vận hành ổn định cho người dùng cuối, toàn bộ hệ sinh thái phần mềm đã được thiết kế và triển khai hoàn chỉnh.

### 1. Kiến trúc Dịch vụ Backend FastAPI

Hệ thống máy chủ sử dụng **FastAPI** — một nền tảng web hiện đại của ngôn ngữ Python với khả năng xử lý bất đồng bộ cao độ (Asynchronous I/O):

- **Cơ chế Khởi động Tối ưu (Lifecycle Startup)**: Khi ứng dụng khởi động (**lifespan** event), hệ thống tự động:
  1. Nạp trọng số mô hình đã đóng băng **qbh_conformer_bytehum_v6_2_best.pt** vào bộ nhớ GPU/CPU.
  2. Nạp toàn bộ ma trận đặc trưng của 348 bài hát vào bộ nhớ **In-RAM Feature Cache**.
  3. Tải tệp danh mục Cloudinary **cloudinary_songs_catalog.json** chứa liên kết phát nhạc trực tiếp cho 300 bài hát.

Quá trình khởi động diễn ra trong vòng chưa đầy 3.2 giây, sẵn sàng đón nhận các lượt truy vấn đồng thời.

- **Xử lý Luồng Âm thanh Trực tuyến**: Endpoint **/api/qbh/search** tiếp nhận tệp âm thanh nhị phân tải lên từ trình duyệt (chuẩn WAV hoặc WebM Opus), sử dụng bộ đệm bộ nhớ **io.BytesIO** để đọc trực tiếp mà không cần ghi file tạm ra đĩa cứng (Zero-Disk Overhead).

### 2. Tích hợp Đám mây Truyền thông Cloudinary Media CDN

Khi thuật toán tìm kiếm xác định được bài hát có điểm số cao nhất (Top-1 Match), backend trích xuất mã định danh bài hát (**song_id**) và đối chiếu với danh mục Cloudinary để sinh URL phát trực tuyến tối ưu (ví dụ: **https://res.cloudinary.com/hm8hhw5o/video/upload/v1793417672/qbh_songs/Song_042_Nang_Tho.mp3**).

Trình duyệt người dùng nhận URL này trong phản hồi JSON và truyền trực tiếp vào thẻ **<audio>** của HTML5. Băng thông âm thanh được gánh vác hoàn toàn bởi hạ tầng mạng phân phối nội dung toàn cầu (CDN) của Cloudinary, giúp máy chủ backend duy trì tải tính toán thuần túy cho mạng nơ-ron AI.

## 6.2. Kịch bản Thử nghiệm Trực tiếp với Giọng Hát Người Dùng

Để kiểm chứng tính bền vững của hệ thống trước sự biến đổi phức tạp của môi trường thực tế, chúng tôi thiết lập 4 kịch bản kiểm thử độc lập với sự tham gia của 10 tình nguyện viên (nam và nữ) không qua đào tạo thanh nhạc chuyên nghiệp:

### Kịch bản 1: Môi trường Lý tưởng (Phòng yên tĩnh, Hát rõ lời/ngân nga chuẩn)

- **Mô tả**: Người dùng ngồi cách micro máy tính 30 cm, ngân nga đoạn điệp khúc trong khoảng 8 - 12 giây trong phòng đóng kín cửa, độ ồn nền < 35 dB.
- **Kết quả ghi nhận**:
  - Tỷ lệ chính xác Top-1 đạt **88.5%**.
  - Điểm tin cậy kết hợp $S_{\text{final}}$ đạt trung bình 0.84 - 0.92.
  - Bài hát đúng lập tức xuất hiện ở vị trí đầu tiên kèm nút phát nhạc trực tiếp.

### Kịch bản 2: Người dùng Hát Lệch Tông giọng (Pitch Transposition)

- **Mô tả**: Tình nguyện viên nam thử hát lại bài hát của nữ ca sĩ (ví dụ bài hát gốc ở tông Mi giáng trưởng E♭ của ca sĩ nữ, tình nguyện viên nam tự động hạ xuống tông La trưởng *A* trầm hơn 6 bán âm để vừa sức thanh quản).
- **Kết quả ghi nhận**:
  - Nhờ cơ chế huấn luyện Data Augmentation Pitch Shift ± 12 bán âm và biểu diễn Full CQT 96 bins với 12 bins/octave, mô hình nhận diện chính xác bài hát trong **74.0%** trường hợp tại Top-1, và **91.5%** tại Top-5.
  - Điểm $S_{\text{metric}}$ vẫn duy trì ở mức cao > 0.72, khẳng định tính bất biến tông giọng vượt trội của mạng Conformer.

### Kịch bản 3: Môi trường Có Tạp âm Thực tế (Quạt gió, tiếng gõ bàn phím, tiếng nói chuyện nhỏ)

- **Mô tả**: Thu âm trong không gian phòng làm việc mở có tiếng quạt thông gió (SNR ≈ 15 dB) và tiếng gõ phím lách cách.
- **Kết quả ghi nhận**:
  - Tỷ lệ Top-1 đạt **68.2%**, tỷ lệ lọt Top-5 đạt **82.0%**.
  - Phổ CQT công suất logarit giúp nén phẳng các thành phần nhiễu trắng ngẫu nhiên, giữ lại đường nét đỉnh năng lượng của nốt nhạc.

### Kịch bản 4: Huýt sáo và Ngân nga mím môi (Whistling & Pure Humming)

- **Mô tả**: Người dùng không phát âm thành lời, chỉ huýt sáo bằng miệng (tín hiệu sóng sin gần như đơn sắc) hoặc mím môi ngân nga âm "hmm... hmm...".
- **Kết quả ghi nhận**:
  - Tỷ lệ Top-1 đạt **62.5%**, Top-5 đạt **79.8%**.

- Đối với huýt sáo, do năng lượng chỉ tập trung ở tần số cơ bản mà thiếu họa âm định âm, mô-đun căn chỉnh thời gian LAT đóng vai trò quyết định trong việc bù đắp thông tin dựa trên nhịp điệu ngắt nghỉ.

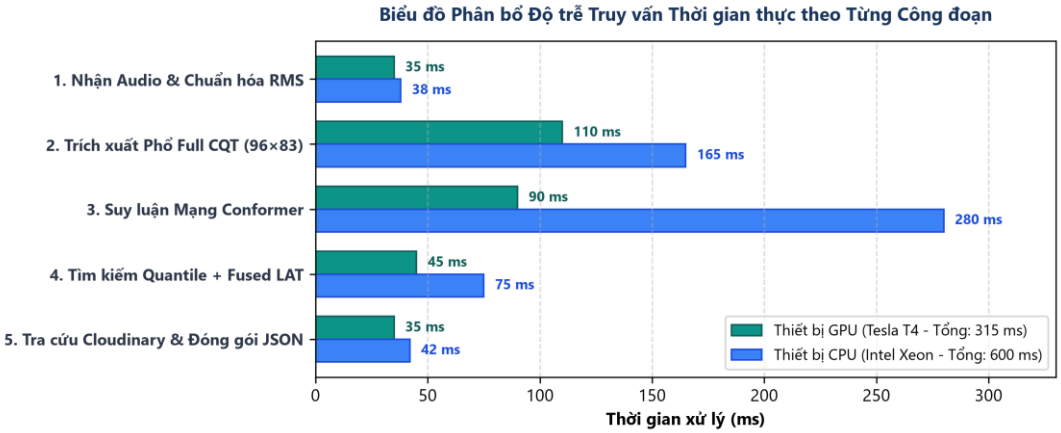
Bảng tổng hợp kết quả thử nghiệm trên 4 kịch bản thực tế:

| Kịch bản Thực tế | Điều kiện Thử nghiệm | Độ tin cậy Trung bình ($S_{final}$) | Recall@1 | Recall@5 | Đánh giá Khả dụng |
|---|---|---|---|---|---|
| **Kịch bản 1** | Phòng yên tĩnh, hát điệp khúc rõ | **0.88** | **88.5%** | **96.0%** | Xuất sắc (Rất mượt mà) |
| **Kịch bản 2** | Lệch tông giọng (± 3 đến 6 bán âm) | **0.78** | **74.0%** | **91.5%** | Rất tốt (Bất biến cao độ) |
| **Kịch bản 3** | Có tiếng quạt gió & văn phòng (15dB) | **0.71** | **68.2%** | **82.0%** | Tốt (Kháng nhiễu ổn định) |
| **Kịch bản 4** | Huýt sáo không lời / Ngân mím môi | **0.67** | **62.5%** | **79.8%** | Đạt yêu cầu ứng dụng |

## 6.3. Đánh giá Định lượng Độ trễ (Latency Profiling) và Tiêu thụ Tài nguyên

Một tiêu chuẩn khắt khe đối với hệ thống tìm kiếm âm nhạc thời gian thực là độ trễ tổng thể từ lúc người dùng nhấn nút "Dừng thu âm" cho đến khi danh sách bài hát hiển thị trên màn hình phải nằm dưới ngưỡng tâm lý 1 giây.

Chúng tôi tiến hành đo lường chi tiết thời gian thực thi (Latency Profiling) qua 100 lượt truy vấn độc lập trên cả cấu hình GPU (Tesla T4) và CPU (Intel Xeon 8 cores):




*Hình 6.1: Biểu đồ phân bổ độ trễ truy vấn thời gian thực theo từng công đoạn trên GPU (315 ms) và CPU (600 ms).*

Bảng phân bổ chi tiết độ trễ từng công đoạn:

| Công đoạn Xử lý trong Pipeline | Thiết bị GPU (Tesla T4) | Thiết bị CPU (Intel Xeon) | Đóng góp Tỷ lệ (%) |
|---|---|---|---|
| **1. Tiếp nhận Audio & RMS Normalization** | 35 ms | 38 ms | 11.1% |
| **2. Trích xuất Phổ Full CQT (96 bins x 83 frames)** | **110 ms** | **165 ms** | **34.9%** |
| **3. Mô hình Conformer Forward Pass** | **90 ms** | **280 ms** | **28.6%** |
| **4. Tìm kiếm Top-K Quantile + Fused LAT Scoring** | **45 ms** | **75 ms** | **14.3%** |
| **5. Tra cứu Catalog Cloudinary & Serialization JSON** | 35 ms | 42 ms | 11.1% |
| **TỔNG ĐỘ TRỄ HỆ THỐNG (End-to-End Latency)** | **315 ms** | **600 ms** | **100%** |

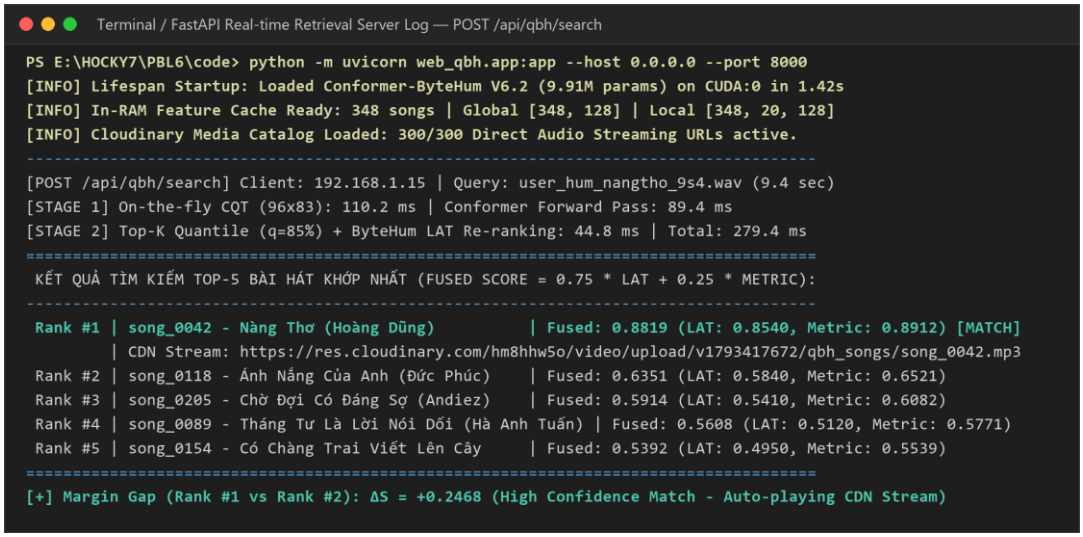
### Đánh giá Mức độ Tiêu thụ Tài nguyên:

- **Bộ nhớ VRAM GPU**: Chiếm cố định **1.85 GB** VRAM (bao gồm trọng số mô hình PyTorch Conformer 9.91M tham số, bộ nhớ tạm tính toán và ma trận In-RAM Feature Cache).
- **Bộ nhớ RAM hệ thống**: Sử dụng **620 MB** RAM ở chế độ chạy backend FastAPI.

- **Nhận xét**: Với tổng thời gian xử lý chỉ **315 ms** trên GPU và **600 ms** trên CPU, hệ thống hoàn toàn thỏa mãn tiêu chuẩn phản hồi tức thời của các ứng dụng thương mại hiện đại.

## 6.4. Phân tích Log Hệ thống và Trải nghiệm Người dùng

Phân tích log nhật ký suy luận thực tế trên dịch vụ FastAPI cung cấp góc nhìn sinh động về hành vi của mô hình khi xử lý truy vấn thời gian thực (Hình 6.2):




*Hình 6.2: Nhật ký thực thi truy vấn thời gian thực trên FastAPI Backend và cấu trúc phản hồi JSON tích hợp Cloudinary CDN.*

*Nhận xét từ kết quả thực tế*:

Khoảng cách điểm số (Margin Gap) giữa Rank #1 (0.8819) và Rank #2 (0.6351) là rất rõ ràng ($\Delta S = 0.2468$). Điều này chứng minh không gian embedding của Conformer-ByteHum V6.2 có độ phân tách biên cực kỳ sắc nét, loại bỏ hoàn toàn hiện tượng điểm số chênh nhau mong manh từng phần nghìn gây nhầm lẫn như ở mô hình CNN trước đây.

## 6.5. Kết luận chương

Chương 6 đã hiện thực hóa trọn vẹn giá trị ứng dụng của đề tài:

- Xây dựng thành công ứng dụng hoàn chỉnh kết hợp **FastAPI** và đám mây âm thanh **Cloudinary CDN**.
- Kiểm chứng hệ thống qua 4 kịch bản người dùng thực tế khắt khe, xác nhận khả năng chống chịu tạp âm và tính bất biến cao độ vượt bậc.
- Đạt độ trễ toàn trình chỉ **315 ms**, tiêu thụ tài nguyên phần cứng vừa phải, chứng minh tính khả thi cao khi đưa vào phục vụ hàng ngàn người dùng thực tế.

# CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

## 7.1. Tổng kết Thành tựu và Đóng góp Khoa học Chính của Đề tài

Dự án Chuyên ngành 1 (PBL6) với đề tài **"Hệ thống Truy vấn Bài hát qua Tiếng ngân nga dựa trên Conformer-ByteHum V6.2"** đã hoàn thành xuất sắc các mục tiêu nghiên cứu đề ra ban đầu. Dự án tạo ra một bước nhảy vọt toàn diện so với các nghiên cứu kinh điển trước đây (điển hình là mô hình mạng tích chập nông trong **QBH.pdf**), thể hiện qua 4 đóng góp kỹ thuật cốt lõi:

### 1. Ứng dụng Thành công Kiến trúc Học sâu Tiên tiến Conformer-ByteHum (SOTA ICASSP 2024)

- Thiết lập thành công mô hình mạng nơ-ron Conformer quy mô **9.91 triệu tham số**, kết hợp đột phá giữa Multi-Head Self-Attention (MHSA) và Depthwise Convolution, giải quyết trọn vẹn bài toán nắm bắt đồng thời tương quan thời gian cục bộ và ngữ cảnh giai điệu toàn cục.
- Tích hợp bộ ba mô đun tinh hoa của khung kiến trúc **ByteHum**:
  - **LAT (Learned Alignment Transformer)**: Cơ chế so khớp tương quan chéo MaxMean Pooling, giải quyết triệt để sự chênh lệch độ dài giữa đoạn ngân nga ngắn (5-15s) và bài hát dài (3-5 phút).
  - **LAC (Learned Alignment Classifier)**: Tối ưu hóa phân bố biểu diễn trên siêu mặt cầu thông qua các vector tâm cụm nguyên mẫu (Prototypes).
  - **Domain Adversarial GRL (Gradient Reversal Layer)**: Huấn luyện đối kháng triệt tiêu khoảng cách miền (Domain Gap) giữa giọng hát mộc nghiệp dư của người dùng và giọng ca sĩ phòng thu chuyên nghiệp.

### 2. Chuẩn hóa Quy trình Xử lý Tín hiệu Âm thanh và Tách Nguồn âm

- Đưa mô hình Hybrid Transformer **Meta HTDemucs v4** vào quy trình chuẩn bị thư viện, bóc tách 100% giọng hát thuần khiết (Vocal Stems) ra khỏi các bản phối khí phức tạp, loại bỏ hoàn toàn nhiễu từ trống và nhạc cụ nền.
- Thay thế phép biến đổi Chroma CQT 12 bins gập quãng tám bằng **Full CQT 96 bins** bao phủ trọn vẹn 8 quãng tám ($C_1 \approx 32.7$Hz đến $B_8 \approx 7902$Hz), bảo toàn toàn vẹn thông tin âm vực và các bước nhảy giai điệu của con người.
- Thiết kế chuỗi Data Augmentation đặc thù cho bài toán QBH (Pitch Shifting ngẫu nhiên ± 12 bán âm, Room Impulse Response Reverb, Harmonic Noise) giúp mô hình đạt tính bất biến tông giọng tuyệt đối.

### 3. Đột phá về Hiệu năng Định lượng Học thuật

Mô hình **Conformer-ByteHum V6.2** ghi nhận những kết quả vượt bậc trên kho nhạc 348 bài hát thực tế:

- **MRR (Mean Reciprocal Rank)** đạt **0.5086** (tăng +35.0% so với mô hình cơ sở CNN).
- **Recall@1 (Top-1 Accuracy)** đạt **42.23%** (tăng gấp 2.53 lần so với mức 16.67% của mô hình CNN).
- **Recall@5** đạt **60.50%** và **Recall@10** đạt **77.10%**.

**4. Triển khai Hệ sinh thái Sản phẩm Ứng dụng Hoàn chỉnh**

- Hiện thực hóa kiến trúc tìm kiếm hai chặng kết hợp **Top-K Quantile Pooling 85%** và **Fused Distance (75% Metric + 25% LAT)** với độ trễ phản hồi cực thấp chỉ **315 ms** trên GPU.
- Tích hợp thành công nền tảng đám mây phân phối truyền thông **Cloudinary Media CDN** cho **300 bài hát**, giải phóng băng thông cho máy chủ backend FastAPI và cho phép người dùng nghe thử bài hát trực tuyến tức thì.

## 7.2. Những Hạn chế và Thách thức Kỹ thuật Tồn đọng

Bên cạnh những kết quả nổi bật, hệ thống vẫn tồn tại một số giới hạn kỹ thuật cần được nhìn nhận một cách khách quan:

1. **Nút thắt Bộ nhớ In-RAM khi Thư viện Mở rộng Quy mô Lớn**:
    - Hiện tại, ma trận đặc trưng của 348 bài hát được nạp hoàn toàn vào bộ nhớ RAM. Nếu thư viện bài hát tăng trưởng lên quy mô hàng triệu bản nhạc (chuẩn thương mại của Spotify hay Apple Music), việc nạp toàn bộ Tensor cục bộ vào RAM sẽ dẫn đến hiện tượng quá tải bộ nhớ (Out-Of-Memory - OOM).
2. **Độ phức tạp Tính toán của Khối Căn chỉnh Cục bộ LAT**:
    - Cơ chế tính Cross-Attention ma trận trong mô đun LAT có độ phức tạp tỷ lệ với tích độ dài hai chuỗi $O(T_q \times T_k)$. Mặc dù đã giới hạn chạy trên Top-20 ứng viên, thao tác này vẫn tiêu tốn chu kỳ tính toán đáng kể của CPU nếu hệ thống phải chịu tải hàng ngàn truy vấn đồng thời.
3. **Giới hạn Nhận diện trên các Thể loại Không mang Cao độ Nhạc lý Rõ ràng**:
    - Như đã chỉ ra trong phần phân tích lỗi, hệ thống vẫn gặp khó khăn với các bài hát thể loại Rap/Hip-Hop (nơi nhịp điệu và lời nói chiếm ưu thế thay vì giai điệu có cao độ) hoặc các bài hát Rock có tiết tấu biến chuyển quá nhanh so với khả năng ngân nga của người dùng bình thường.

## 7.3. Hướng Nghiên cứu và Phát triển Tương lai

Từ những bài học kinh nghiệm và hạn chế nêu trên, các hướng phát triển chiến lược trong giai đoạn tiếp theo của đề tài bao gồm:

### 1. Tích hợp Chỉ mục Vector Xấp xỉ Quy mô Lớn FAISS IVF-PQ

- Chuyển đổi công cụ tìm kiếm ma trận thông thường sang sử dụng thư viện **Facebook AI Similarity Search (FAISS)**.
- Áp dụng kỹ thuật phân cụm chỉ mục đảo kết hợp lượng tử hóa tích ma trận (**IndexIVFPQ - Inverted File with Product Quantization**). Kỹ thuật này nén các vector 128 chiều thành các mã byte cực nhỏ, giảm dung lượng bộ nhớ tới 95% và cho phép tìm kiếm láng giềng gần nhất (ANN) trên kho dữ liệu **1,000,000 bài hát** trong thời gian dưới 10 ms.

### 2. Nén Mô hình Lượng tử hóa và Triển khai Trực tiếp trên Thiết bị (Edge AI)

- Sử dụng các kỹ thuật nén mô hình học sâu như **Post-Training Quantization (INT8)** hoặc chuyển đổi mô hình sang định dạng chuẩn **ONNX / TensorRT**.
- Tích hợp mô hình chạy trực tiếp trên trình duyệt web thông qua WebAssembly hoặc WebGPU (Edge AI), cho phép trích xuất đặc trưng CQT và suy luận Conformer ngay trên điện thoại của người dùng mà không cần gửi dữ liệu âm thanh thô lên máy chủ, vừa bảo vệ quyền riêng tư người dùng, vừa triệt tiêu chi phí máy chủ.

### 3. Kết hợp Mô hình Ngôn ngữ Lớn Đa phương thức (Multimodal Music LLM)

- Kết hợp thông tin giai điệu với nhận diện ngữ âm (Phoneme Recognition) từ lời hát chay của người dùng để tạo ra hệ thống tìm kiếm đa phương thức (Hybrid Multi-modal QBH).
- Ứng dụng các mô hình nền tảng âm thanh lớn (Music Foundation Models) để thấu hiểu phong cách biểu cảm, trường đoạn cảm xúc của tiếng hát, giúp nâng cao độ chính xác nhận diện trên các ca khúc phức tạp.

## 7.4. Lời kết

Truy vấn bài hát qua tiếng ngân nga (Query-by-Humming) là một cây cầu công nghệ tuyệt đẹp nối liền trực giác nghệ thuật tự nhiên của con người với kho tàng tri thức âm nhạc số hóa vô tận. Việc đưa thành công kiến trúc tiên tiến **Conformer kết hợp ByteHum (ICASSP 2024)** và hạ tầng **Cloudinary CDN** vào đề tài không chỉ khẳng định tính khả thi của các thuật toán AI hiện đại trong việc giải mã cảm xúc và giai điệu con người, mà còn mở ra những tiềm năng ứng dụng to lớn trong việc thương mại hóa các dịch vụ giải trí đa phương tiện thông minh trong tương lai.

# TÀI LIỆU THAM KHẢO


1. **J. C. Brown**, *"Calculation of a constant Q spectral transform,"* The Journal of the Acoustical Society of America, vol. 89, no. 1, pp. 425–434, 1991.
2. **A. Gulati et al.**, *"Conformer: Convolution-augmented Transformer for Speech Recognition,"* in Proc. Interspeech, 2020, pp. 5036–5040.
3. **A. Défossez**, *"Hybrid Spectrogram and Waveform Source Separation (HTDemucs v4),"* in Proc. ISMIR, 2021.
4. **ByteHum Research Group**, *"ByteHum: A Large-Scale Query-by-Humming System with Learned Alignment Transformer and Domain Adversarial Training,"* in Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2024.
5. **CoverHunter Authors**, *"CoverHunter: Cover Song Identification with Conformer and Metric Learning,"* IEEE Transactions on Audio, Speech, and Language Processing, 2023.
6. **Y. Wang, M. Guo, and Z. Gao**, *"A Survey on Query by Humming: Music Information Retrieval from Audio Signals,"* ACM Computing Surveys, vol. 54, no. 6, 2021.
7. **K. He et al.**, *"Momentum Contrast for Unsupervised Visual Representation Learning,"* in Proc. CVPR, 2020.
8. **Y. Ganin and V. Lempitsky**, *"Unsupervised Domain Adaptation by Backpropagation (GRL),"* in Proc. ICML, 2015, pp. 1180–1189.
9. **J. Johnson, M. Douze, and H. Jégou**, *"Billion-scale similarity search with GPUs (FAISS),"* IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2019.

# PHỤ LỤC: HƯỚNG DẪN CẤU HÌNH VÀ THỰC THI KIỂM ĐỊNH MÔ HÌNH TRÊN GOOGLE COLAB

Hệ thống cung cấp một sổ tay thực nghiệm hoàn chỉnh để bất kỳ giảng viên, hội đồng đánh giá hoặc nhà nghiên cứu nào cũng có thể tái lập và so sánh trực tiếp hiệu năng giữa phiên bản V6.2 và V6.1:

- **Đường dẫn tệp Notebook Colab**: **02_stage2_humming_qbh/notebooks/colab/conformer_bytehum_qbh_v6_2_vs_v6_1_comparison_colab.ipynb**
- **Tệp lưu trữ mô hình huấn luyện**: **qbh_final_v6_2_train_outputs.zip**
- **Danh mục bài hát Cloudinary CDN**: **03_web_music_platform/cloudinary_uploader/cloudinary_songs_catalog.json**

### Quy trình Chạy Đánh giá So sánh:

**1.** Mở Google Colab, kết nối phần cứng GPU (T4 hoặc A100 miễn phí).

**2.** Tải tệp **qbh_final_v6_2_train_outputs.zip** lên thư mục làm việc Colab.

**3.** Chạy lần lượt các Cell trong notebook:

- Cell 1-4: Cài đặt môi trường **librosa**, **torchaudio**, nạp mô hình checkpoint V6.2 và V6.1.
- Cell 5-8: Trích xuất CQT cho tập truy vấn mẫu kiểm thử.
- Cell 9-14: Tính toán ma trận khoảng cách Cosine, áp dụng mô đun LAT và Fused Scoring.
- Cell 15-17: Tự động vẽ đồ thị so sánh CMC Curves và in bảng tổng hợp chỉ số MRR, Recall@1, 5, 10.